

Announcements

- Requirements Assignment Due this Sunday
- Keep in mind that Code Review Assignment is also due with Project 1
 - PRs should be well spaced in time for full credit
- Mid semester Feedback Survey Released
 - Please express your comments/concerns about the course so far.
 - Your responses will be stored anonymously
 - Due Oct 7
- Guest Lecture this Friday
- Extra Credit Based on this lecture in next Wednesday's class



CS3300 Introduction to Software Engineering

Lecture 10: Software Architecture; UML Design

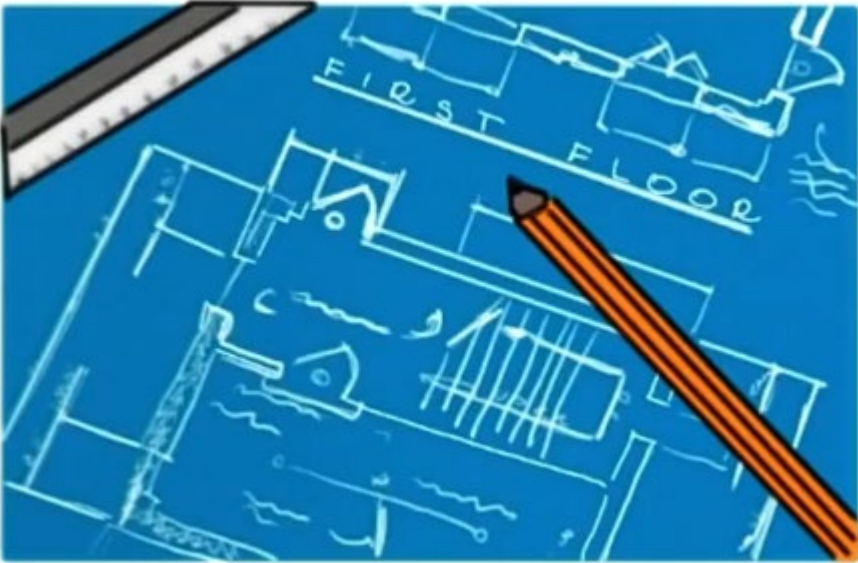
Dr. Nimisha Roy ▶ nroy9@gatech.edu

Software Architecture

- The fundamental **organization of a system**, embodied in its components, their **relationships to each other** and the environment, and the principles governing its **design** and evolution. [ANSI/IEEE Std 1471-2000]
- Expert developers' understanding of the system design. [Ralph Johnson]

A general definition of SWA

Set of principal design decisions about the system



Blueprint of a software system

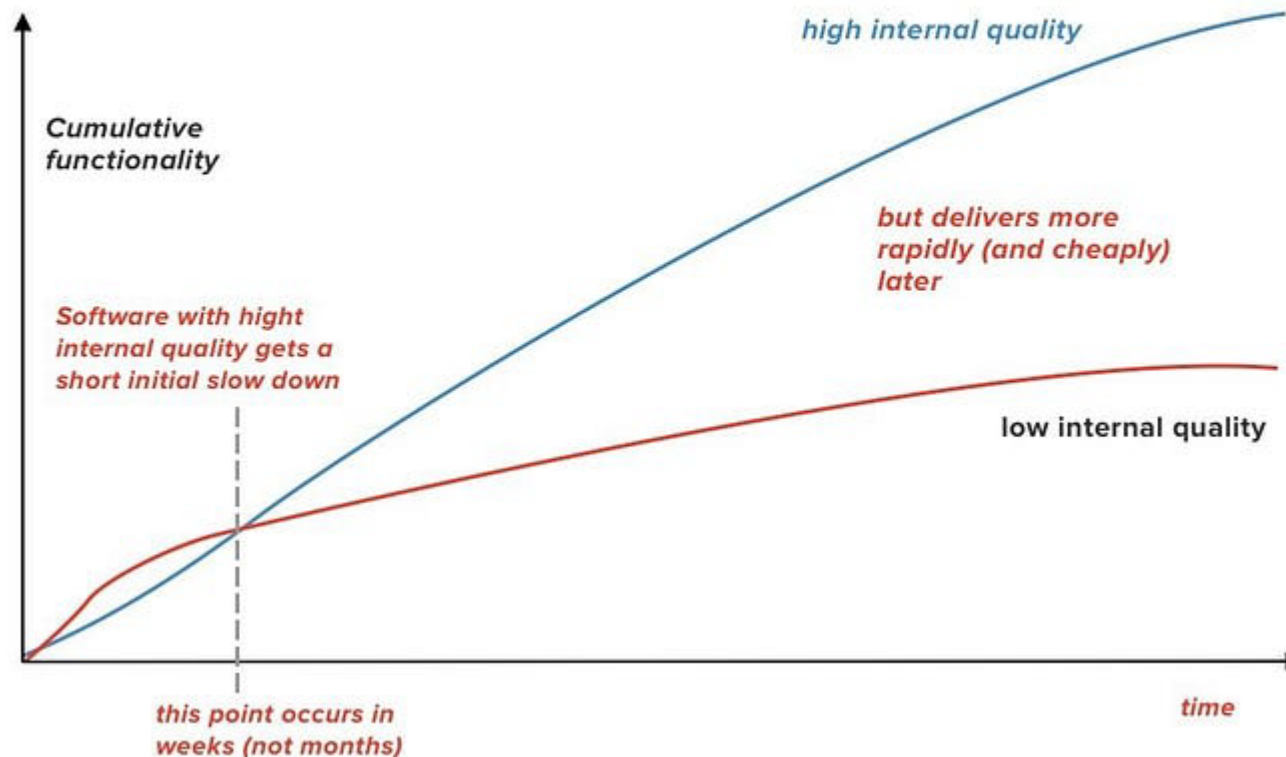
- Structure
- Behavior
- Interaction
- Follow business requirements

In Netflix, it's their microservice architecture that empowers them to manage availability, whereas, in Salesforce or Google, it's a domain-driven design that helps them run domain logic complexity.

Why is SWA important?

- Helps in maintaining high-quality code, facilitating future changes, and enhancing the longevity of the system.
- Crucial for managing complex systems at scale, enabling smooth feature integration over time.

Difference Between Low Quality High Quality Software

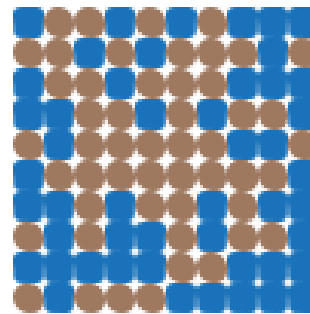


Why is SWA important?

Tricky subject for the clients of software products - as it isn't something they immediately perceive. Poor architecture is a major contributor to the growth of cruft - elements of the software that impede the ability of developers to understand the software.

- Software that contains a lot of cruft is much harder to modify, leading to features that arrive more slowly and with more defects.
- High internal quality leads to faster delivery of new features

If we compare one system with a lot of cruft...

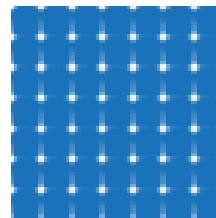


the cruft means new features take longer to build



this extra time and effort is the cost of the cruft, paid with each new feature

...to an equivalent one without



free of cruft, features can be added more quickly

Why is SWA important?

Scenario: Suppose two similar products have been launched within the gap of a month. After three months, they require new features to be added.

- Launch on May 31: Messy, tangled code leads to quicker launch but complicates later changes.
- Launch on June 30: Clean, well-architected code is easier to maintain but leads to a slightly delayed launch.

What would a software development company choose to do?

Usually, despite the messy code blocks, team would go for an earlier launch → that's what would matter for the time being - quicker launch → better opportunities to monopolize market. In the second scenario, the changes would take time as quality performance and quality code have been given equal importance → This would disturb time-to-market unfavorably. But, a well-defined system architecture design in the form of microservices will help in easier maintenance. → your company saves time, but it will also satisfy the users with fast and regular updates for new features.

A well-defined architecture allows for easier maintenance and consistent updates, saving time and delivering timely results.

Temporal Aspect



SWA evolves over time.

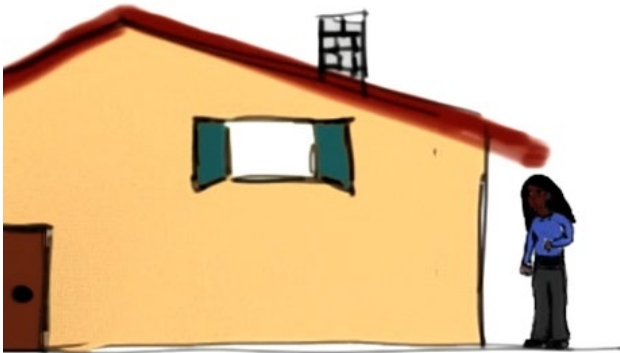
At any point in time, there is a SWA, but it will change over time

Design decisions are made, unmade, and changed over a system's lifetime.

Prescriptive vs. Descriptive Architecture



A prescriptive architecture captures the design decisions made prior to the system's construction
=> as- conceived SWA

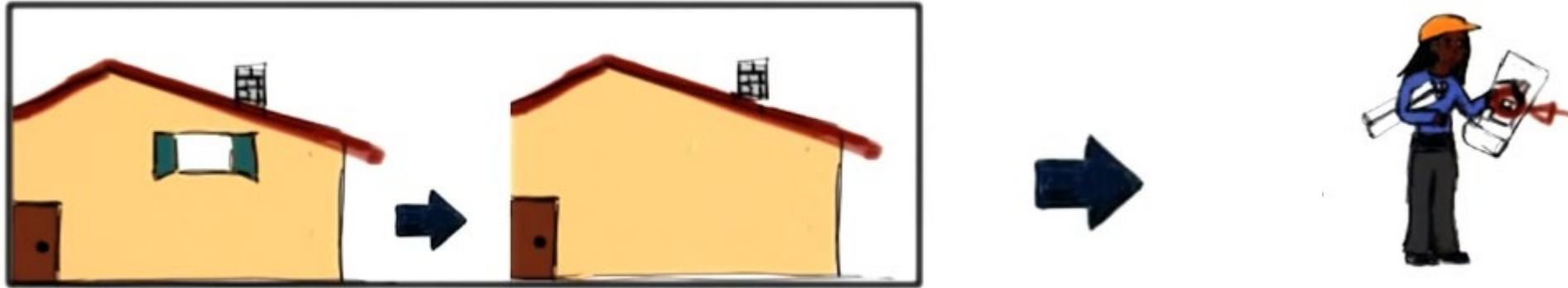


A descriptive architecture describes how the system has actually been built
=> as- implemented SWA

Architectural Evolution



When a system evolves, ideally its prescriptive architecture should be modified first



In practice, this rarely happens

- Developer's sloppiness
- Short deadlines
- Lack of documented prescriptive architectures

Architectural Degradation



Architectural drift : Introduction of architectural design decisions orthogonal to a system's prescriptive architecture



Architectural erosion : Introduction of architectural design decisions that violate a system's prescriptive architecture

Maintaining alignment between prescriptive and descriptive architecture is key to avoiding "drift" and "erosion."

Architectural Recovery

Drift and Erosion => Degraded architecture

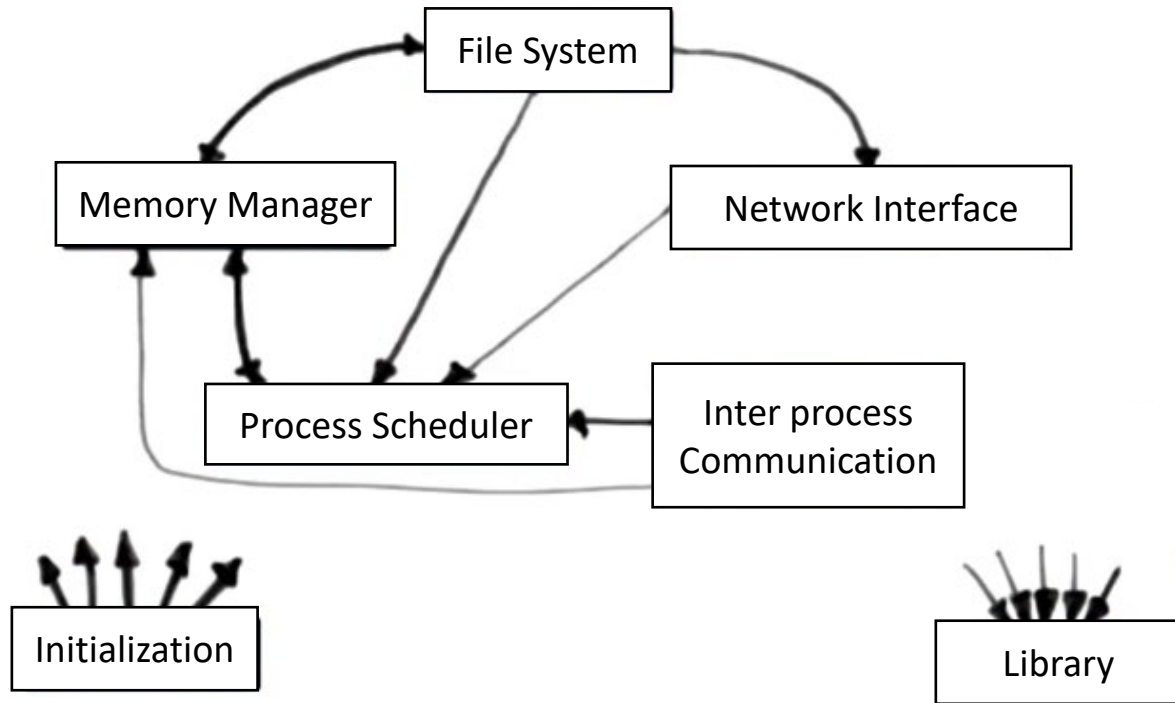


Keep tweaking the code (typically disastrous)

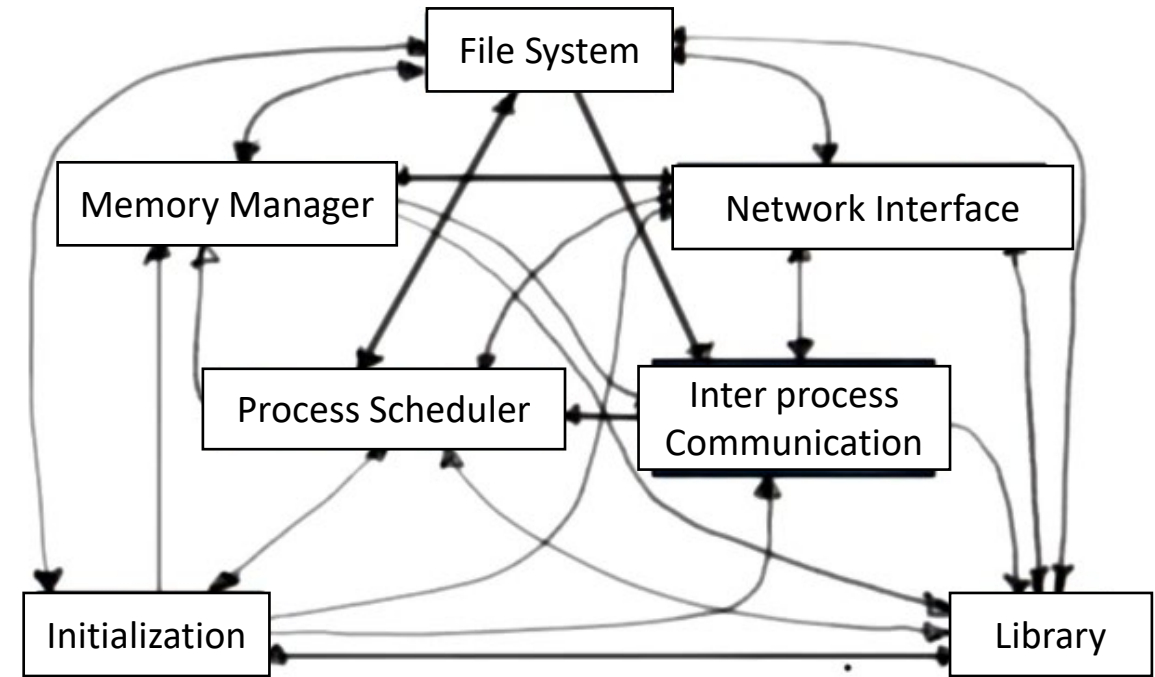


Architectural recovery: determine SWA from implementation and fix it

An example from the Linux Kernel



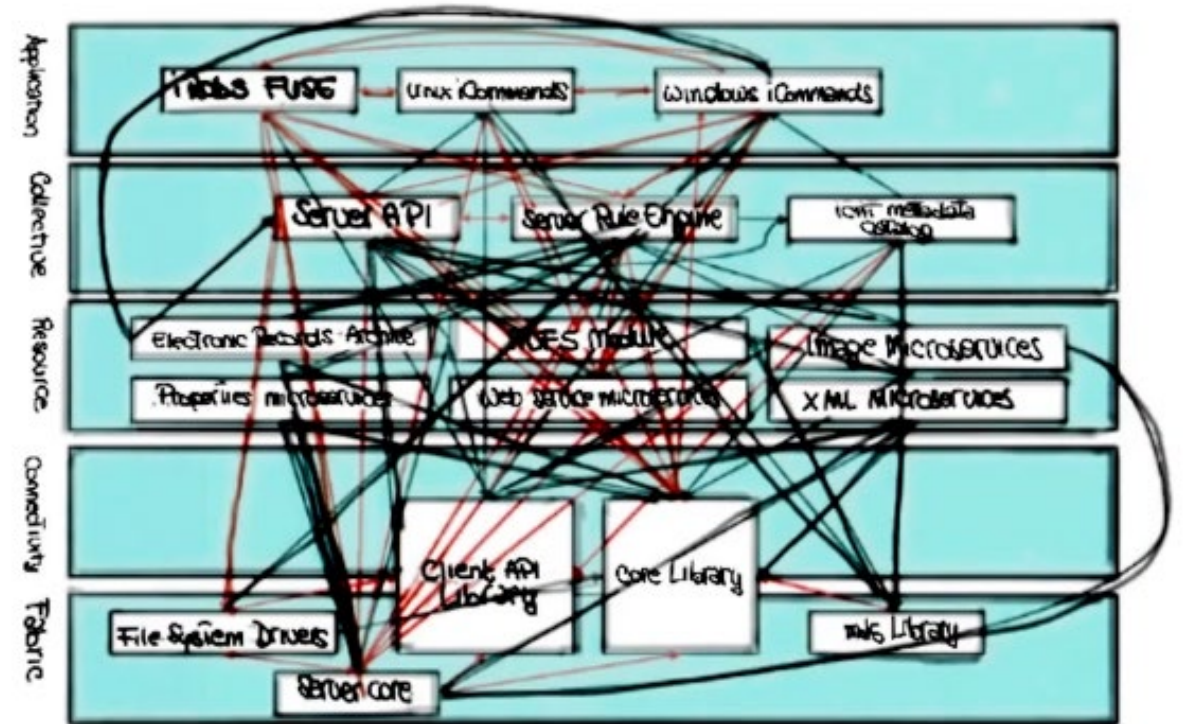
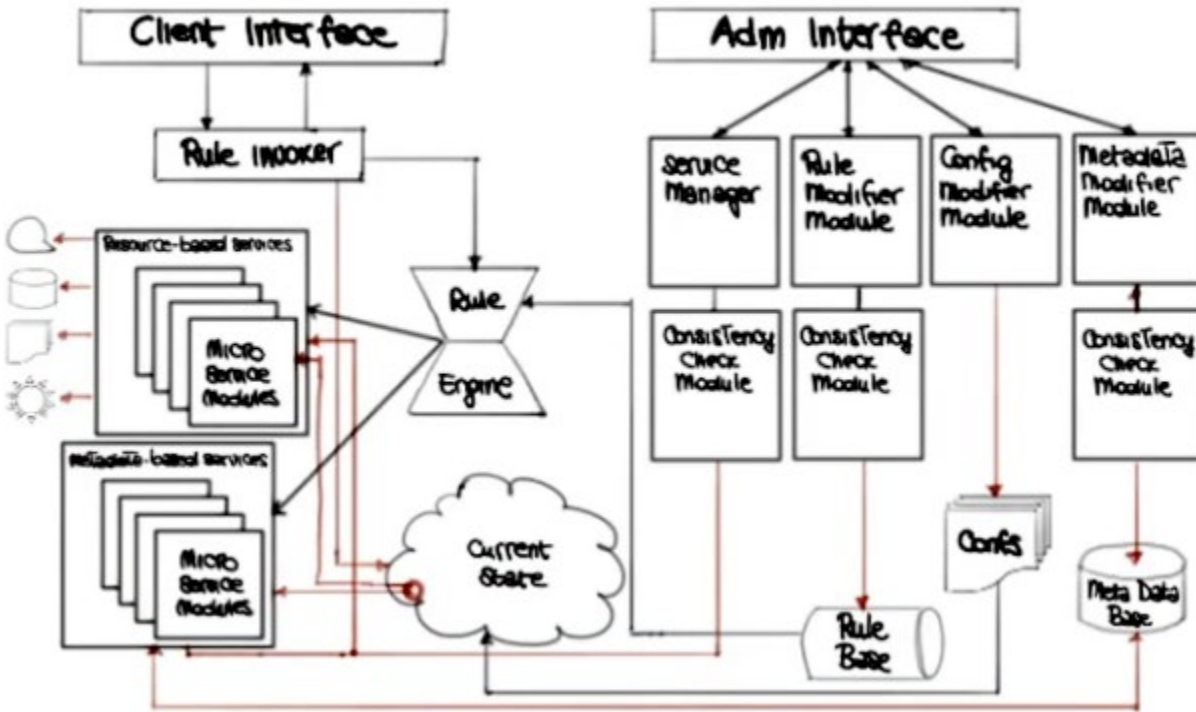
Prescriptive Architecture



Descriptive Architecture

Another example: iRODS

Data grid system that was built by a biologist. It's a system for storing and accessing big data.

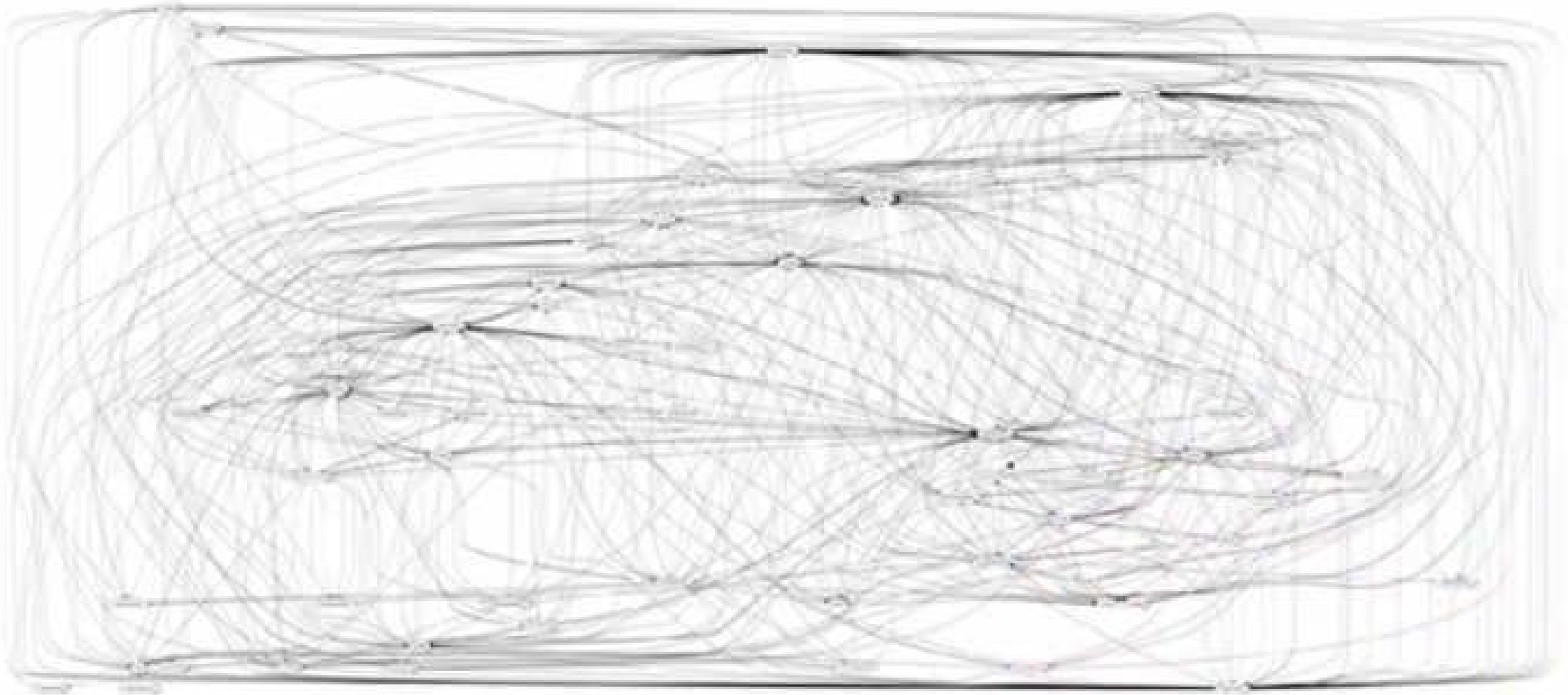


Prescriptive Architecture

Descriptive Architecture

More examples: Hadoop

Open-source software framework for storage and large-scale processing of data sets

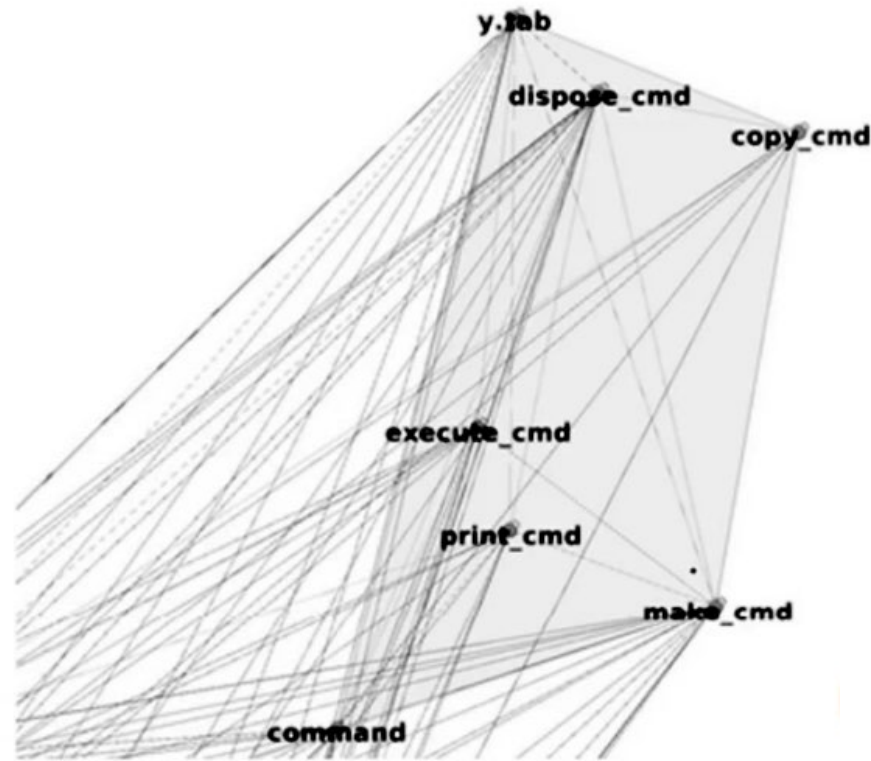


Descriptive Architecture

61 out of the 67 components in the system have circular dependencies, which means that they depend on each other in a circular way and this is normally not a good thing

Final example: Bash

Unix shell written as a free software replacement for the traditional Bourne shell



Lack of cohesion in the component

High coupling among components

Descriptive Architecture of the command component of Bash.

How AI can Help Maintain Architecture

- AI can help with architectural recovery by scanning the current codebase and determining the descriptive architecture.
- AI can automate documentation generation and code reviews, ensuring descriptive architecture evolves alongside system changes.
- AI-based tools monitor system changes and suggest updates to maintain architectural coherence.

Example: AI tools like **DeepCode** help developers avoid architecture degradation by automatically detecting bad practices early in the development cycle.

Architectural Patterns



An architectural pattern defines “a family of systems in terms of a pattern of structural organization; a vocabulary of components and connectors, with constraints on how they can be combined”

M. Shaw and D. Garlan, 1996

Basically, named collection of architectural design decisions applicable in a given context.

Top Architectural Patterns



Layered

Microkernel

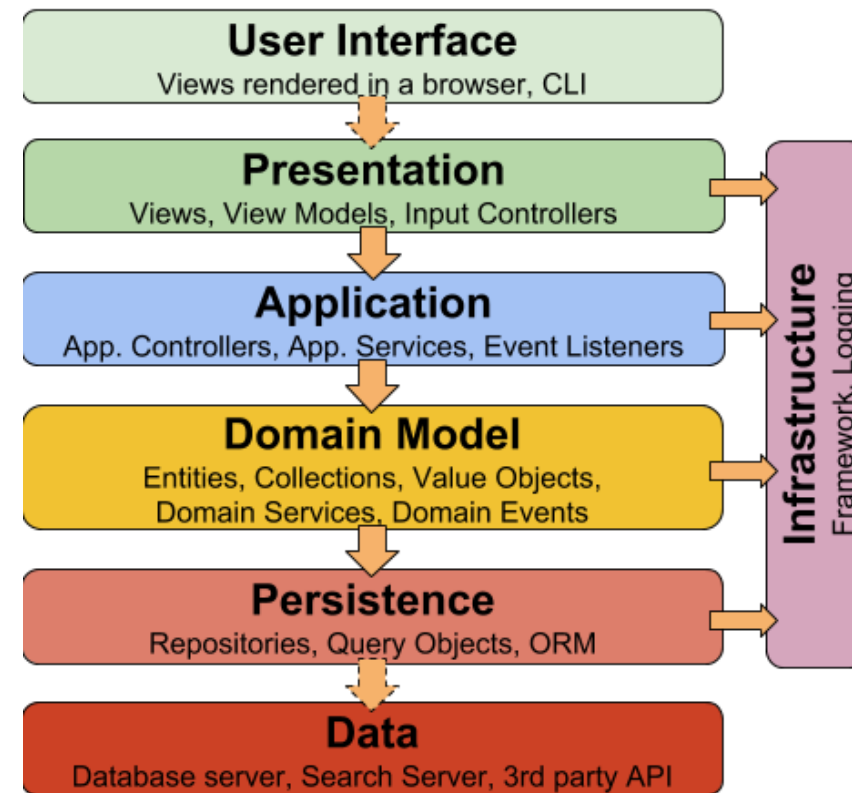
Microservices

Event-Driven

Serverless

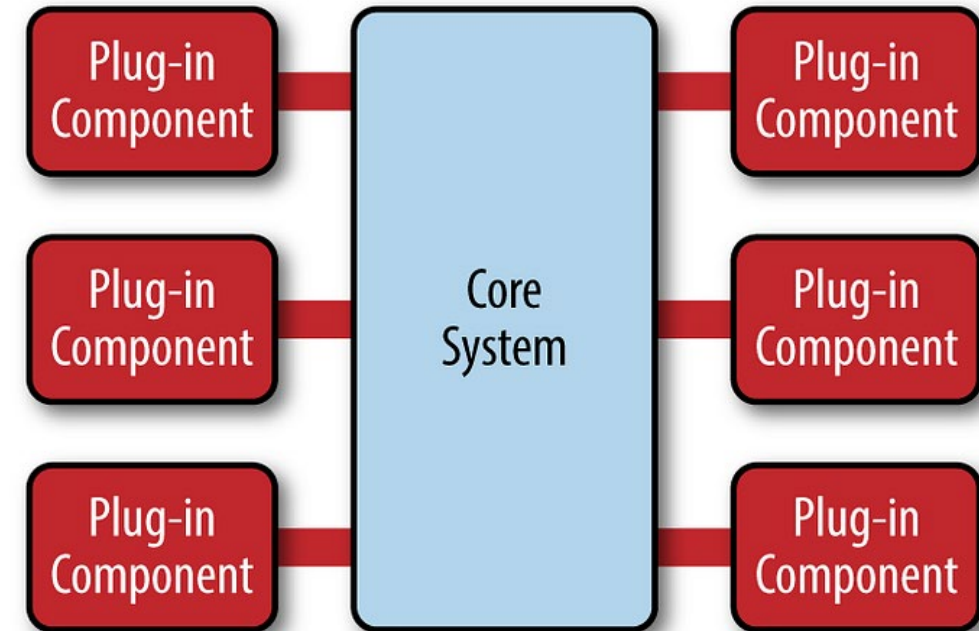
Layered Architecture

- Organizes the system into layers with related functionality/logic associated with each layer
- In a layered system, each layer:
 - Depends on the layers beneath it
 - Is independent of the layers on top of it, having no knowledge of the layers using it
- Example: Wordpress
- Standard usage: Standard apps for quick development with fewer and inexperienced developers; Apps with strict testability and maintainability standards



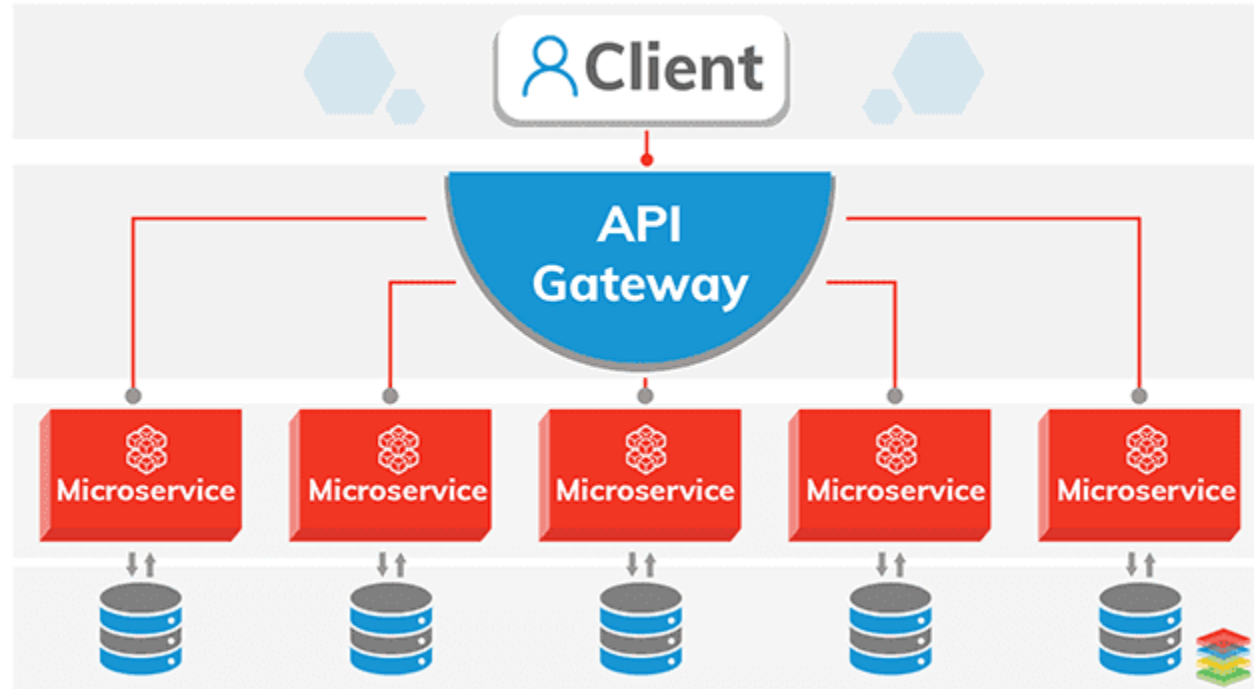
Microkernel Architecture

- separated into an extended functionality (plug-ins) and a minimal functional core comprising standard business logic without any custom code for complex conditional processes or exceptional cases.
- plug-ins consist of independent components that support the core by offering specialized processing added features
- best suited for apps that need to be adaptive and flexible enough to frequently changing system requirements.
- best usage - Task and job scheduling apps, Workflow apps, Apps that incorporate data from various sources, transform the data and send it to different destinations

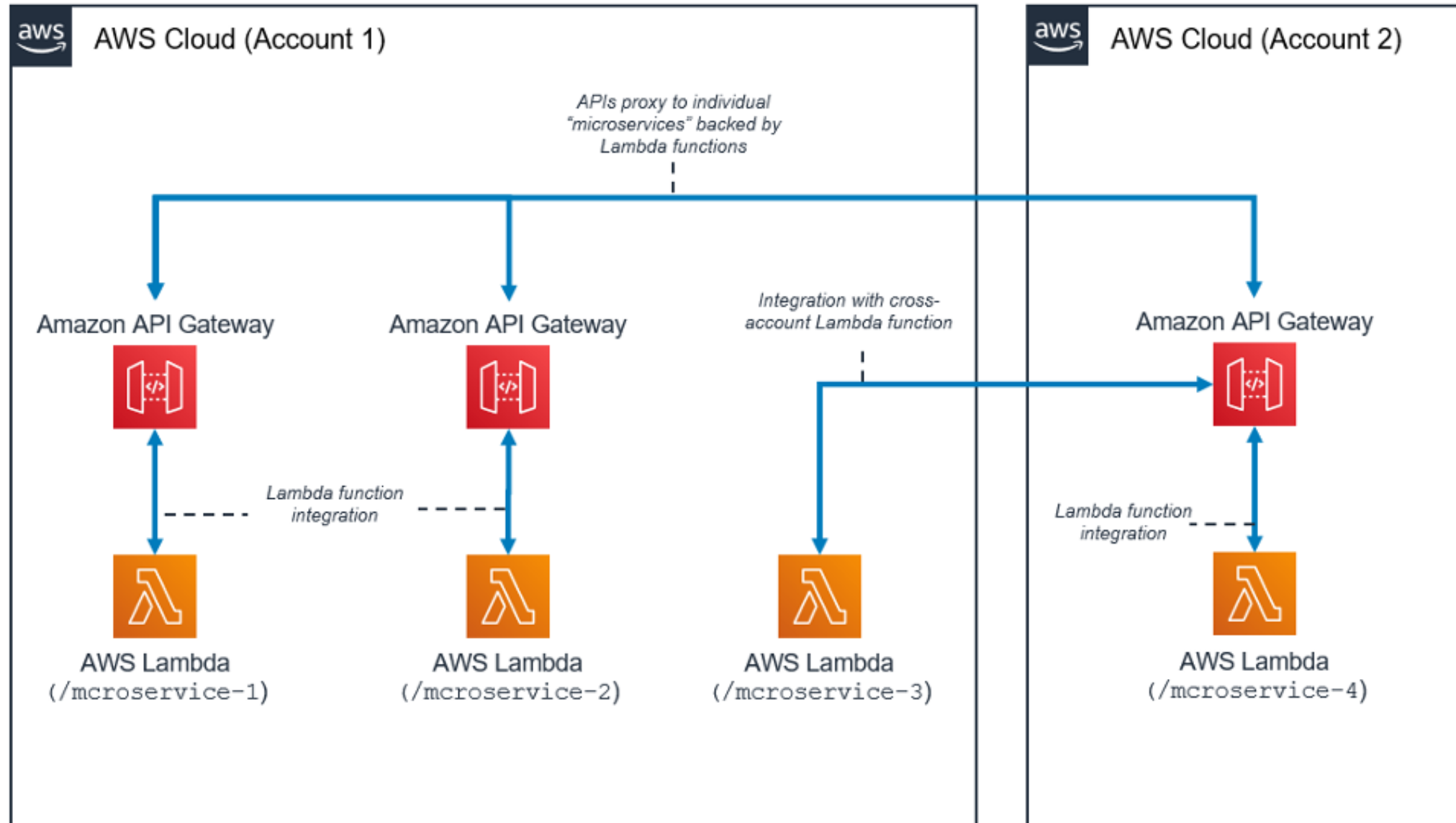


Microservices

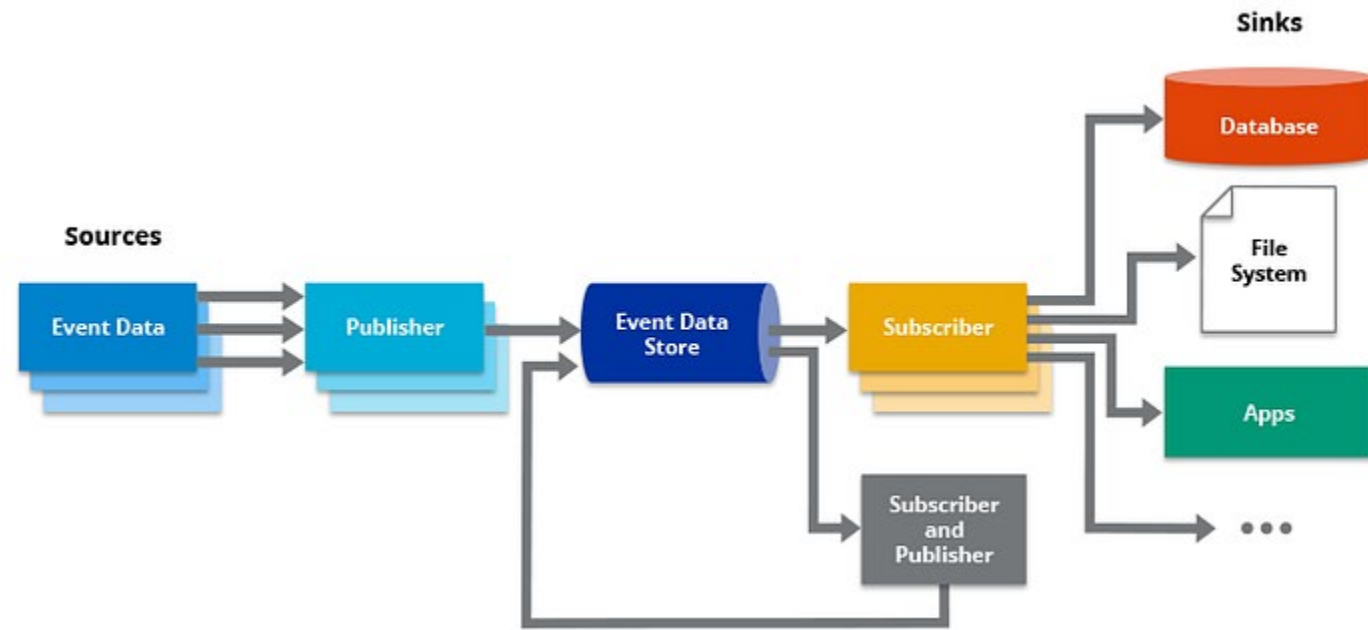
- approaches the building of multiple small and independent apps that work together under an entire system. Each app or microservice has its own responsibility, and the only dependence between them is to communicate.
- Allows easy scalability of development
- Best usage: Fast developing web and business apps; Corporate data hubs with well-established boundaries; Websites with small components



Microservices example: AWS Cloud & Netflix



Event-Driven Architecture (EDA)



- most commonly allocated asynchronous architecture pattern for developing highly scalable systems.
- Its approach is based on data that defines 'events', such as moving the scroll bar, clicking a button, etc., and processes them asynchronously.
- involves single-purpose event processing elements that build a central unit. The central unit then accepts all data and assigns it to separate modules handling the specific type.
- Best usage: User interfaces; Apps that have asynchronous data flow; Complex apps that require seamless data flow and would eventually grow

EDA Example: e-commerce website

A good example is an e-commerce site.

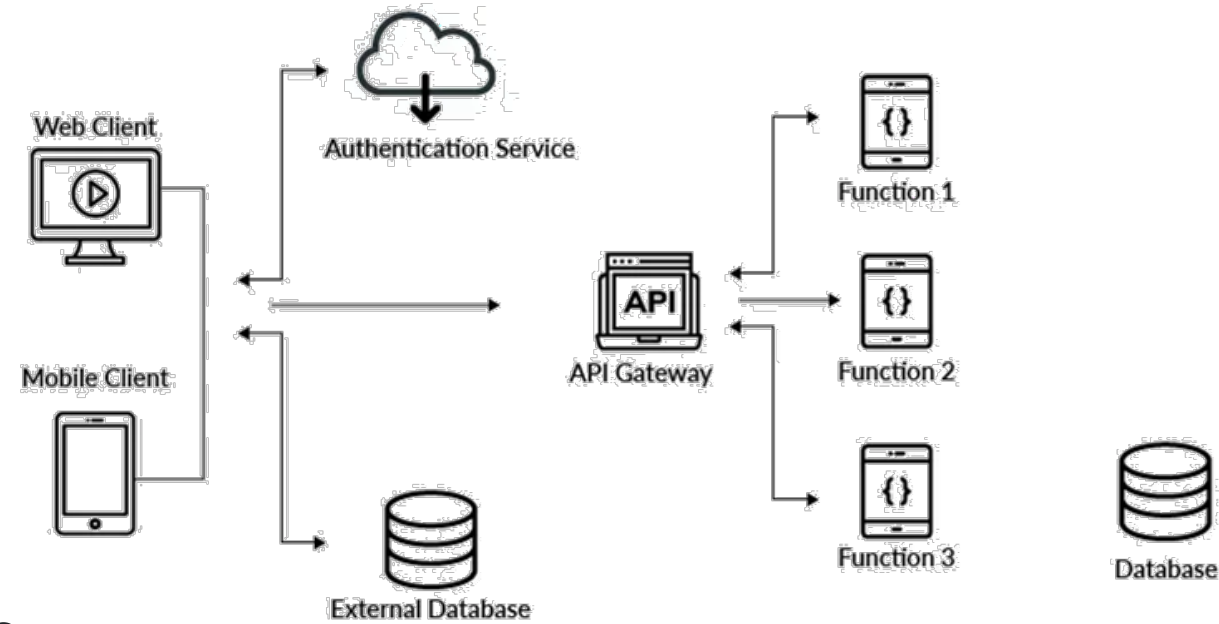
- Enables the e-commerce website to react to various sources at a time of high demand.
 - such as user login, item search, cart modification, order placement, payment, and more.
- Simultaneously, it avoids any crash of the application or any over-provisioning of resources.

Usage:

- For applications where individual data blocks interact with only a few modules.

Serverless Architecture

- Allows developers to build and run services without having to manage the underlying infrastructure.
- Developers can write and deploy code, while a cloud provider provisions servers to run their applications, databases, and storage systems at any scale.
- Serverless functions are triggered by events, such as HTTP requests, database changes, or file uploads. They execute in response to these events, making serverless architecture inherently event-driven and highly responsive.
- E.g. is AWS Lambda, Google Cloud Functions (GAE Standard) and Azure Functions



What makes a good software architecture?

- 1.Functionality:** The extent the software performs against its needed purposes.
- 2.Usability:** The level the software can be used with ease and convenience.
- 3.Reliability:** The capability of the product to provide intended functionality under given circumstances.
- 4.Supportability:** The facility with which developers can transfer the software from one platform to another with minimal or no changes.
- 5.Performance:** The approximation by considering resource utilization, processing speed, response time, productivity, and throughput.
- 6.Self-Reliance:** The capability of independent activities for optimal performance even if one is going through a downtime.

Leveraging AI for Software architecture

Architectural Visualization & Documentation

- AI tools can automatically generate visual representations of complex architectures **from code**, helping developers understand and improve system design - detect bottlenecks, dependencies, or design flaws
- AI-generated architecture diagrams **from prompts (Lucidchart, Eraser.io)** significantly reduces the time spent on tech solution design but also provides architects with a foundational blueprint to kickstart their design.

Real-Time Monitoring

- AI-based tools (**AWS X-ray, Dynatrace**) continuously monitor the system's architecture to detect drift and architectural degradation.
- These tools can provide real-time alerts and recommendations for corrective actions when architectural principles are violated, helping to maintain consistency.

Leveraging AI for Software architecture

Legacy Systems

- AI (**Grok**, **Cast AI**) can help refactor and upgrade legacy systems by identifying outdated patterns and suggesting modern design solutions.
- Legacy applications are often hard to maintain due to their monolithic design.
- AI tools can analyze the code and provide recommendations for migrating to modern architectures like microservices or serverless.

Code Insights

- AI can automatically detect code smells and restructure messy codebases to make them more maintainable and efficient.
- For example, tools like **DeepCode** or **Codex** help refactor code to improve readability and performance.

Leveraging AI for Software architecture

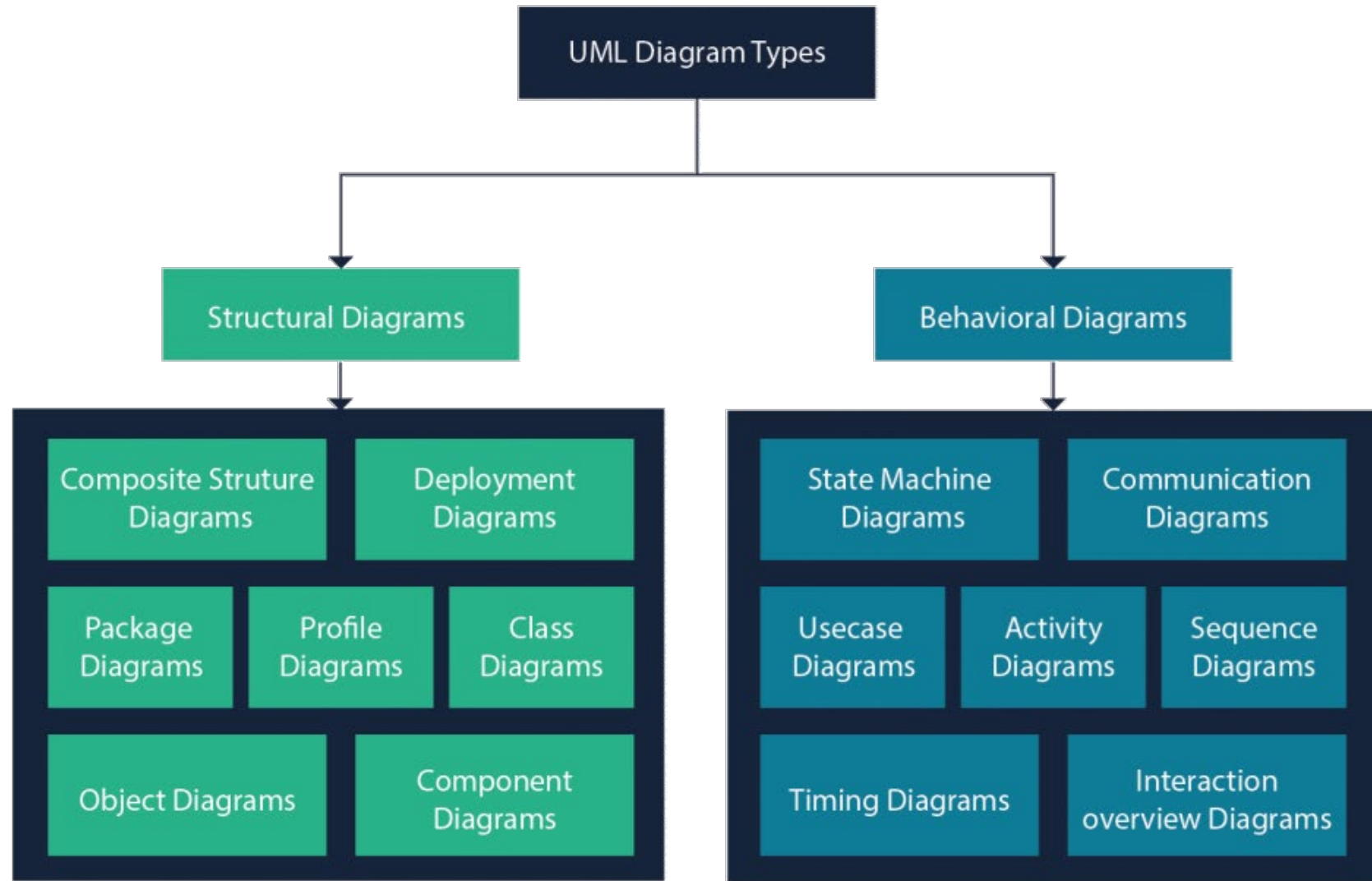
Other insights:

<https://medium.com/inspiredbrilliance/generative-ai-in-software-architecture-dont-replace-your-architects-yet-cde0c5d462c5>

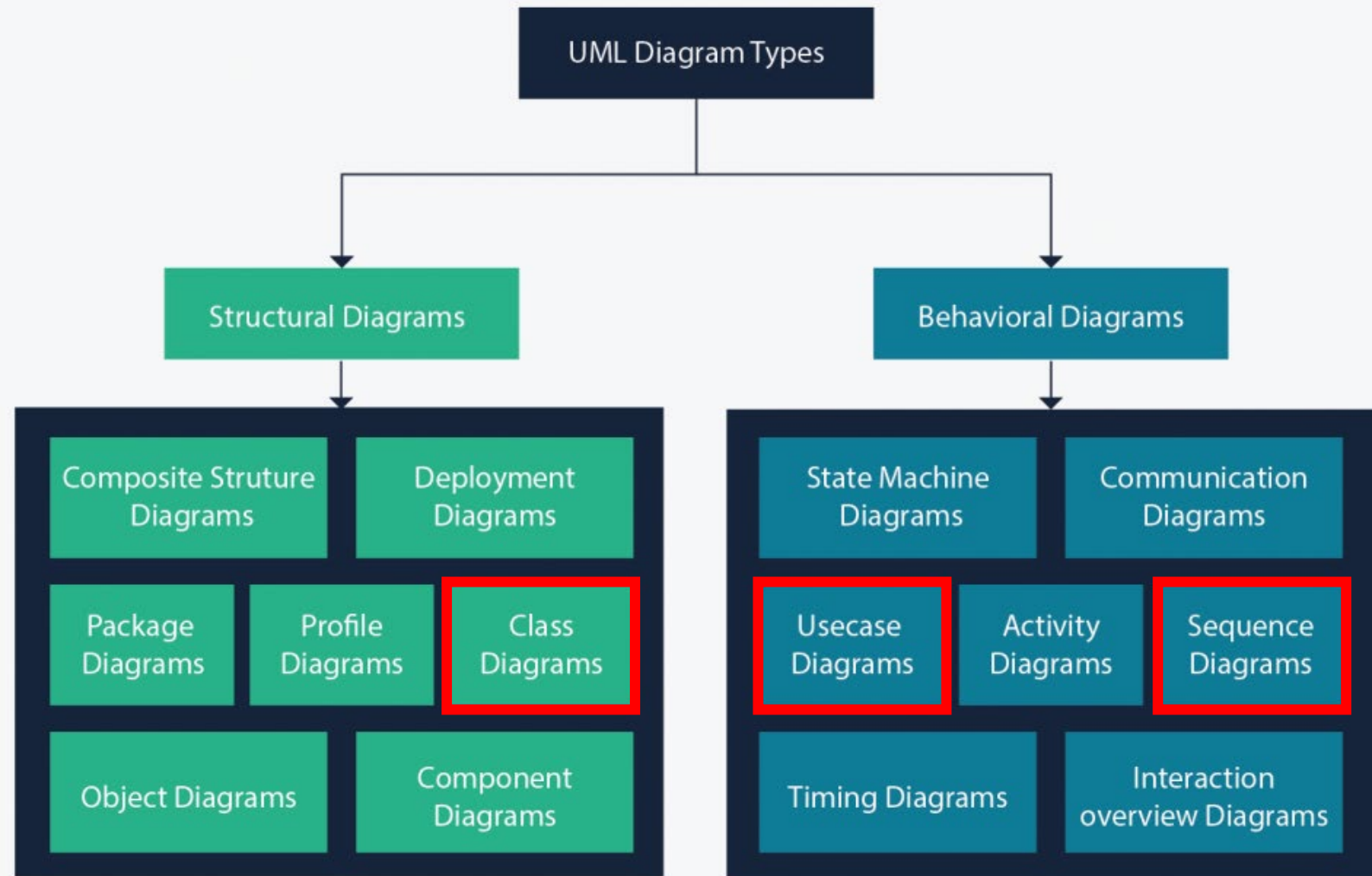
<https://www.linkedin.com/pulse/generative-ai-software-architecture-design-lev-z-/>

Unified Modeling Language

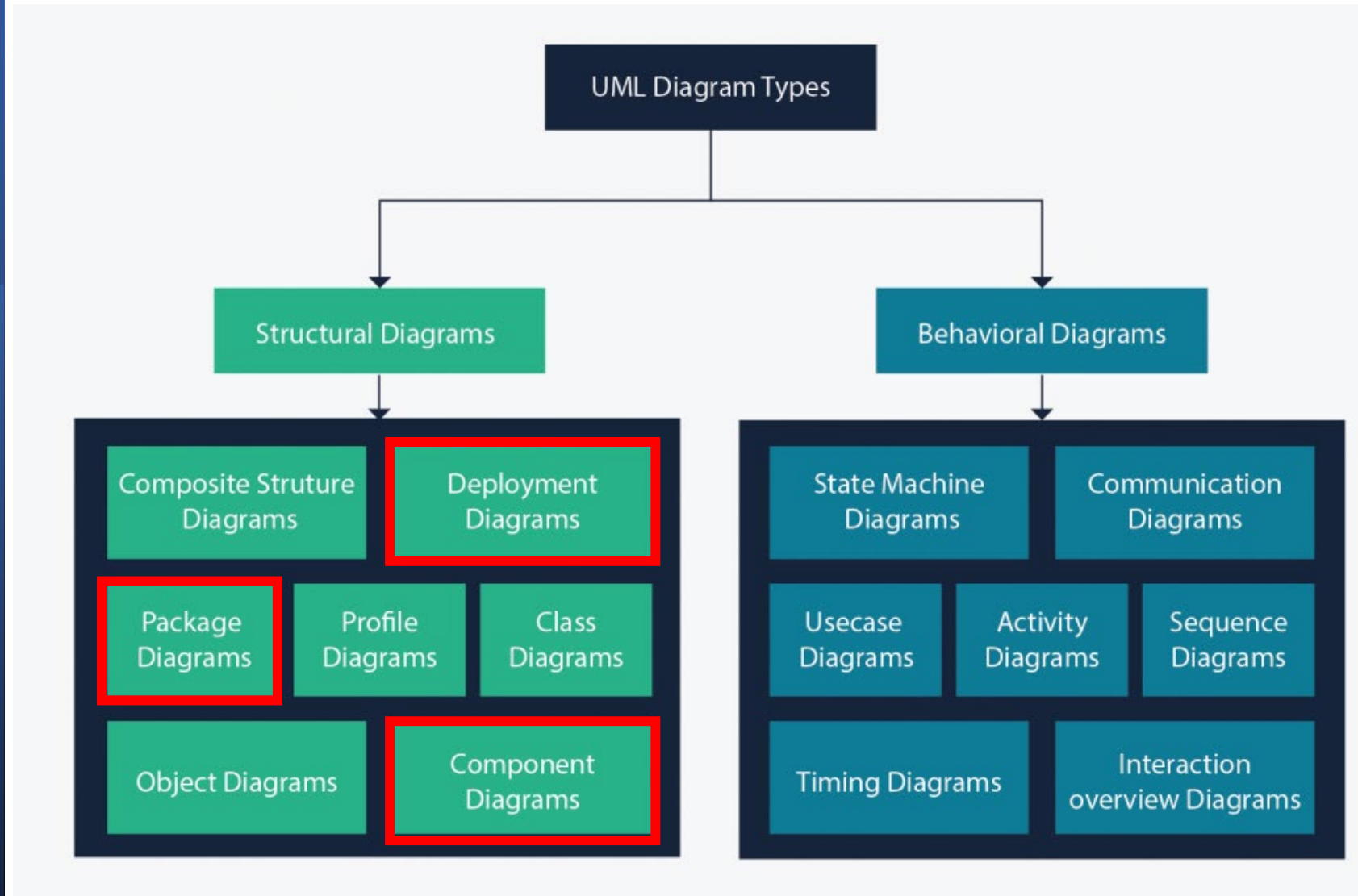
Intended to provide a standard way to visualize the design of a system.



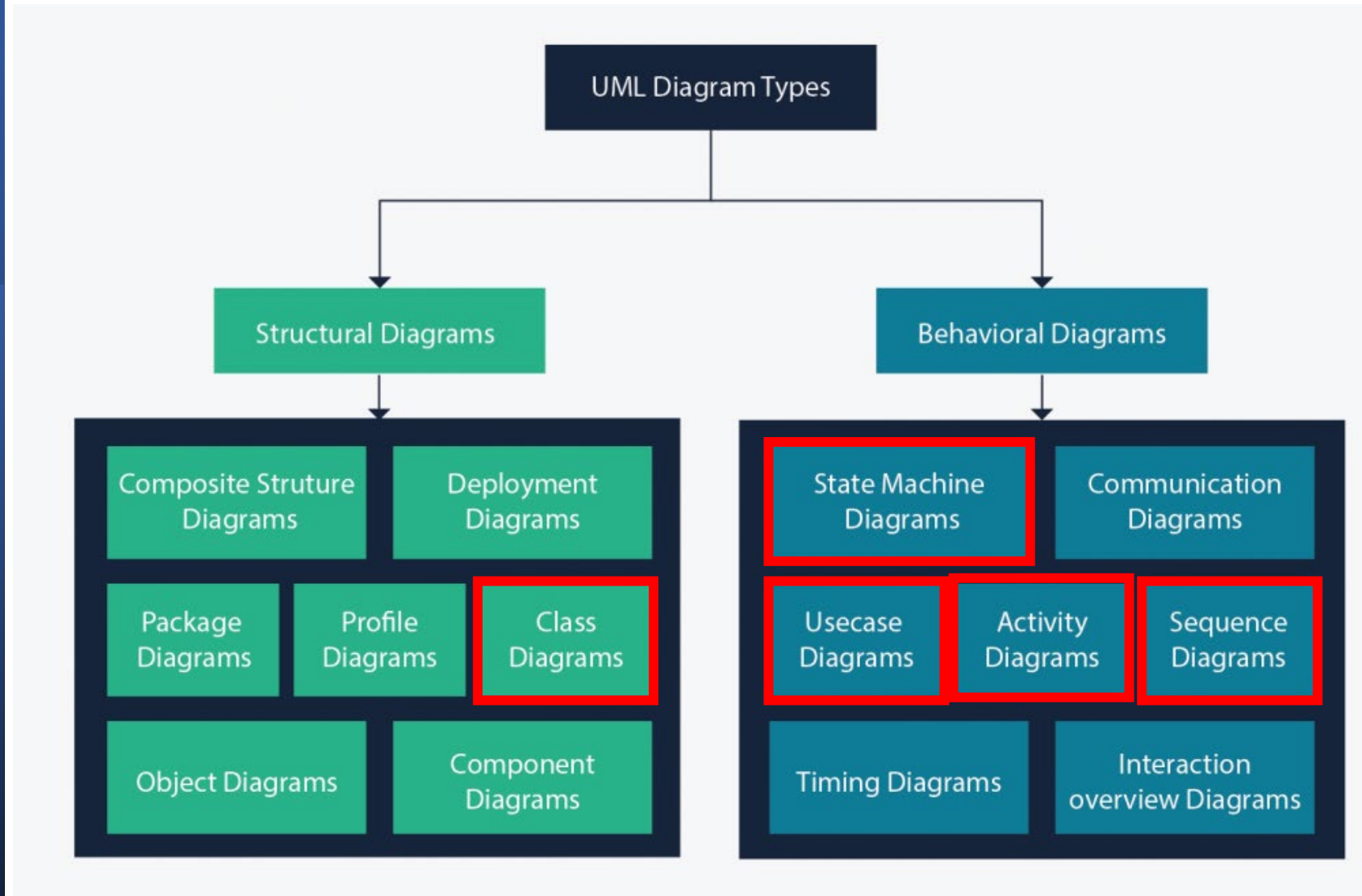
Unified
Modeling
Language: You
may already
know...

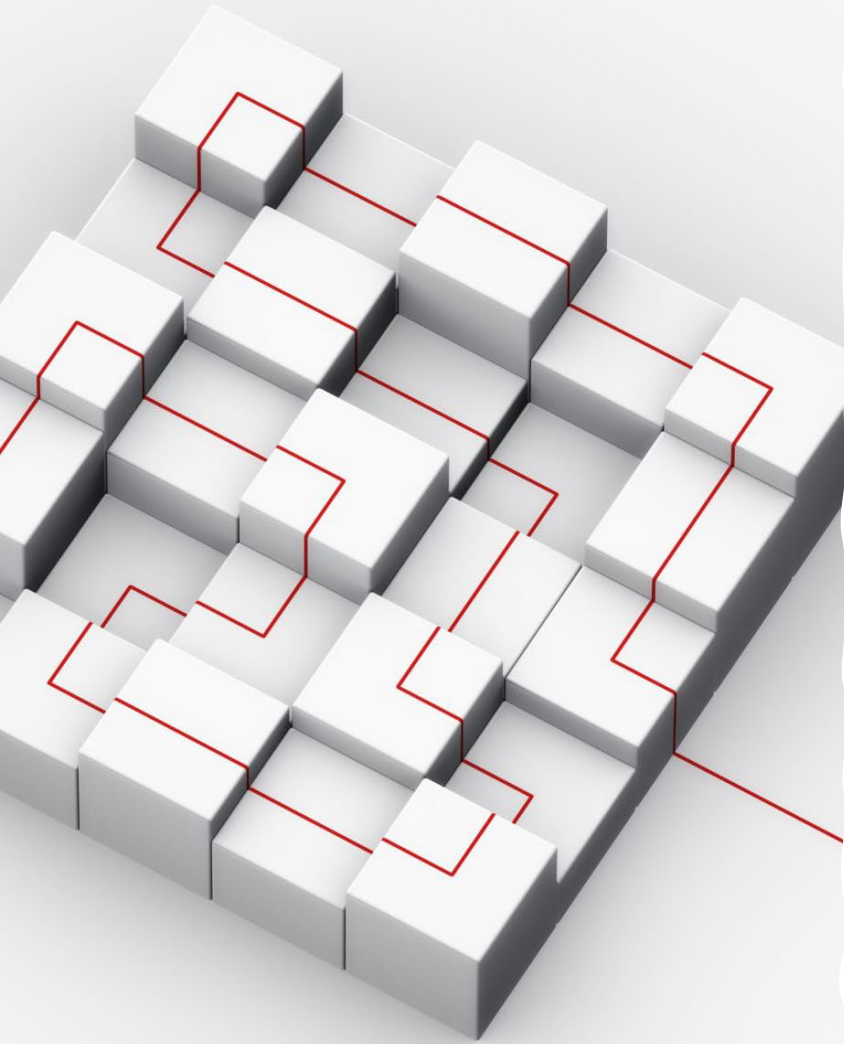


Unified Modeling Language: Architectural View



Unified Modeling Language: Low Level Design View





Architectural View – Structural Diagram: Component

Component Diagram

Static view of components and their relationships

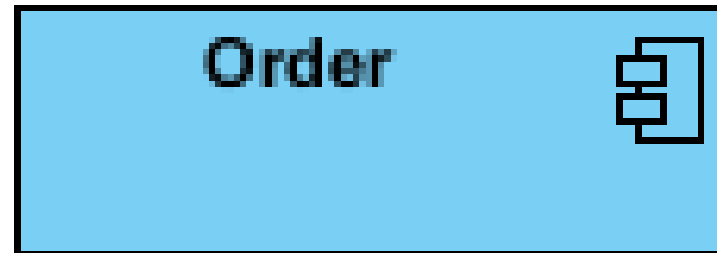
Shows how software components are interconnected and how they will interact with one another, emphasizing the organization and dependencies of different components.

Can be used to represent a software architecture

Component Diagram: Component

Components represented as

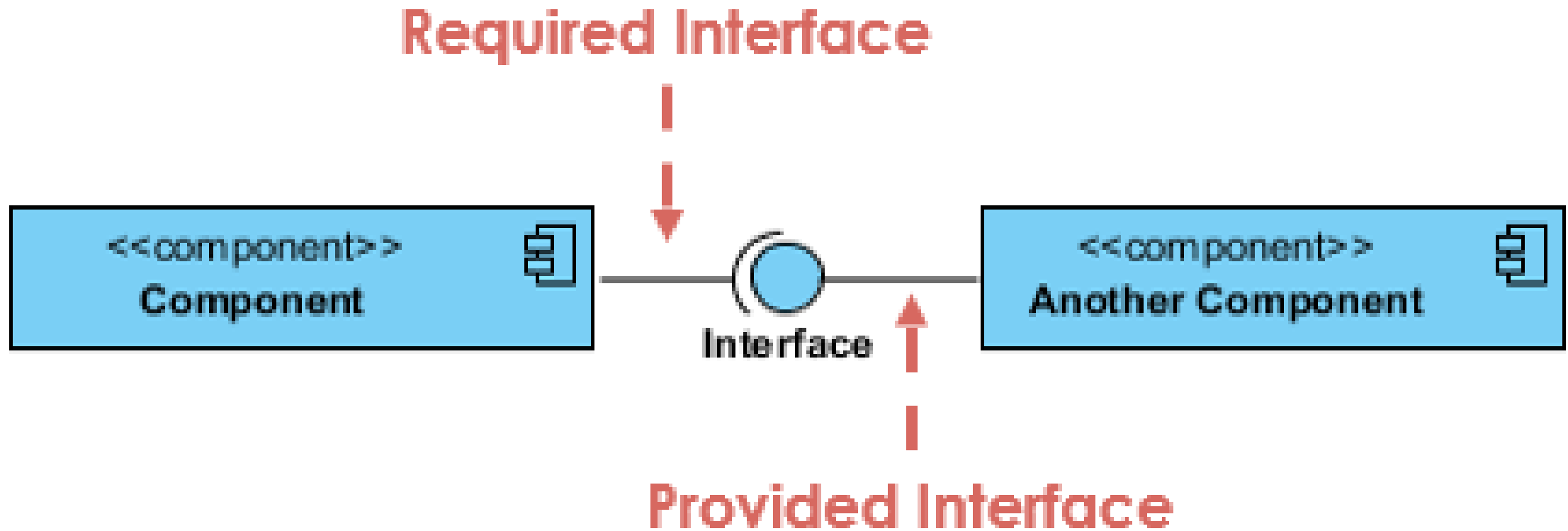
1. A rectangle with the component's name
2. A rectangle with the component icon
3. A rectangle with the stereotype text and/or icon



Component Diagram: Interface

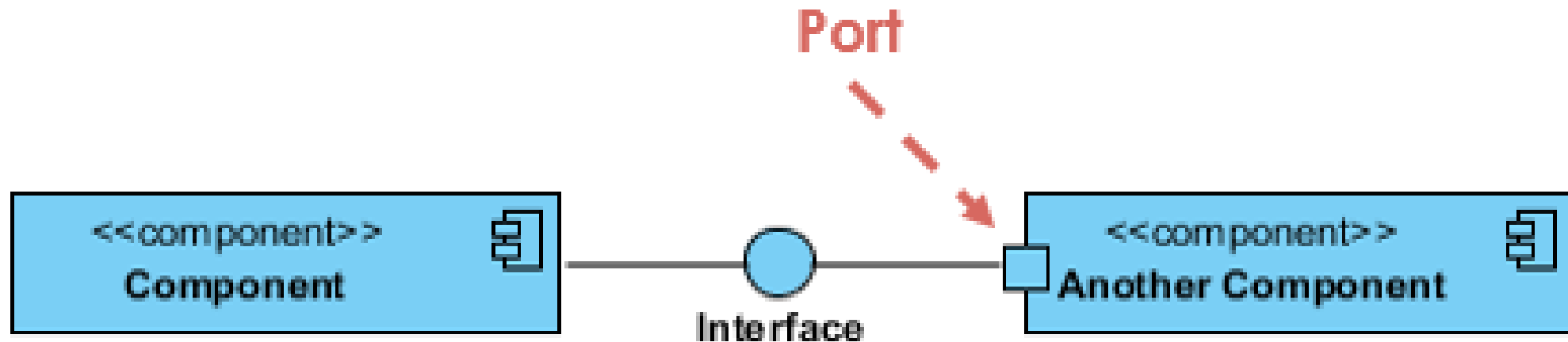
Provided interface symbols with a complete circle at their end represent an interface that the component provides - "lollipop" symbol

Required Interface symbols with only a half circle at their end (a.k.a. sockets) represent an interface that the component requires.



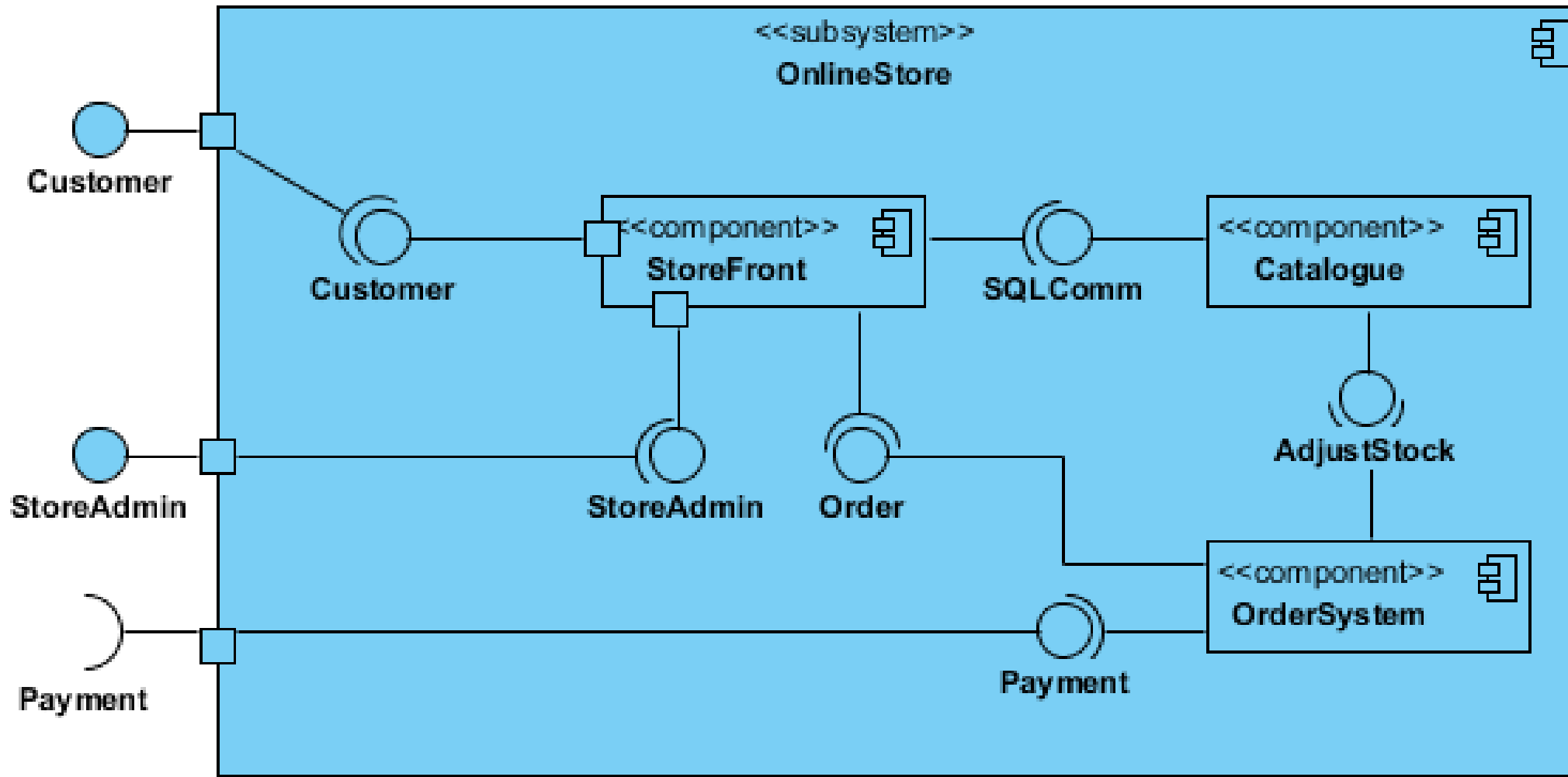
Component Diagram: Port

Ports are represented using a square along the edge of the system or a component. A port is often used to help expose required and provided interfaces of a component.

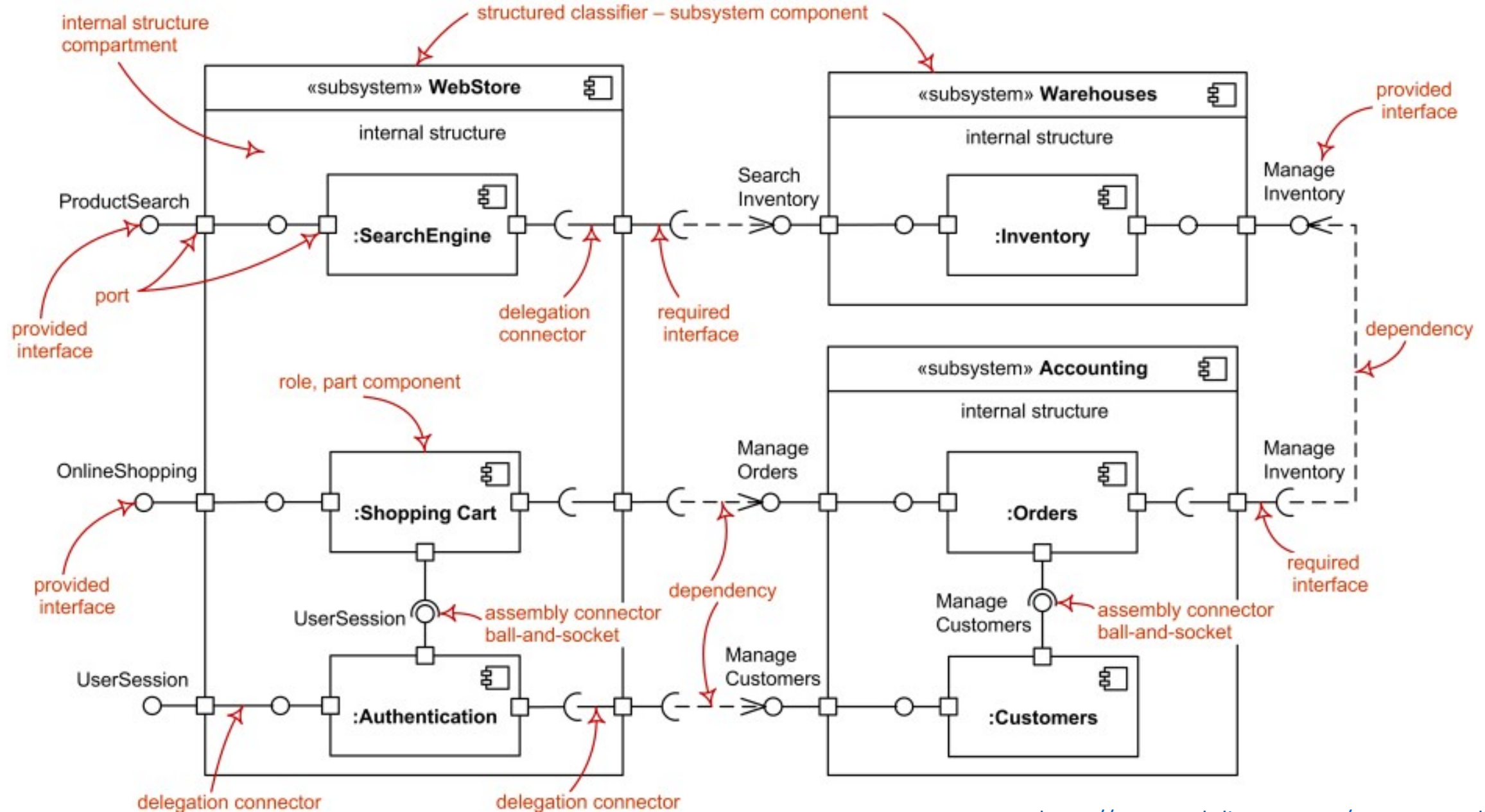


Component Diagram: Subsystem

Collection of components that work together to execute a set of related functionalities and is often used to represent a larger segment of your system.
subsystem notation element has the keyword of <<subsystem>>



Component Diagram Example: ecommerce



Example Quiz



Consider a simplified social media application that consists of the following software components:

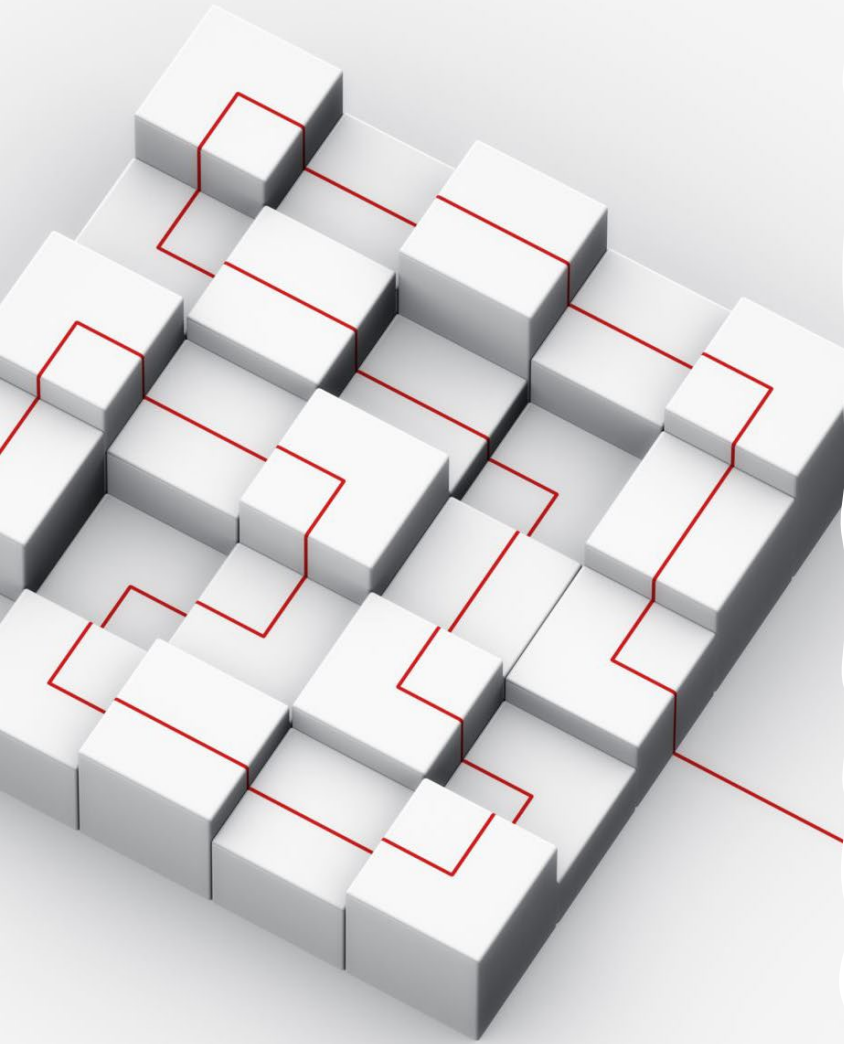
- **User Profile Component:** Manages user information, profile creation, and updates.
- **Post Management Component:** Handles creating, deleting, and editing posts and content.
- **Friendship Management Component:** Manages friend requests, friend lists, and related operations.
- **Messaging Component:** Handles direct messages between users.
- **Notification Component:** Manages notifications related to posts, friend requests, and messages.

Example Quiz



Which of the following statements is likely accurate regarding the dependencies between these components?

- A. The Messaging Component depends on the Post Management Component to send messages.
-  B. The Notification Component depends on the Friendship Management Component to notify users of new friend requests.
- C. The User Profile Component depends on the Messaging Component to update user profiles.
- D. The Post Management Component depends on the User Profile Component to create posts.



Architectural View – Structural Diagram: Deployment

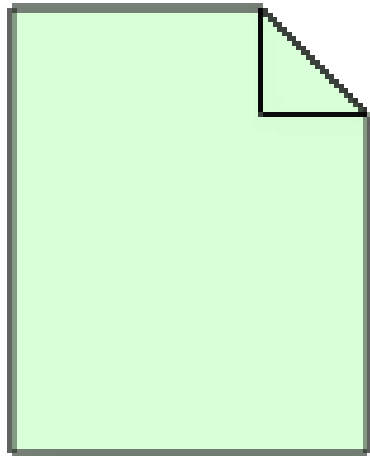
Deployment Diagram

Specifies the physical hardware on which the software system will execute. It also determines how the software is deployed on the underlying hardware.

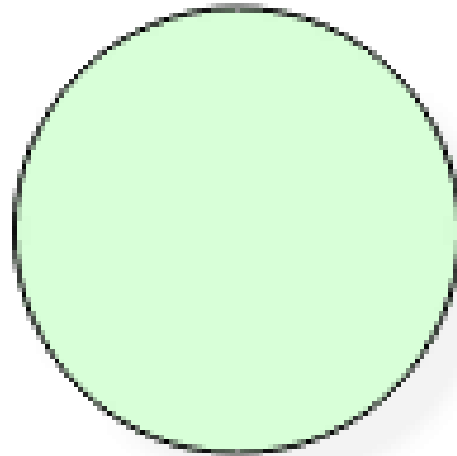
Physical Allocation of components to computational units

Maps the software architecture created in design to the physical system architecture that executes it. In distributed systems, it models the distribution of the software across the physical nodes.

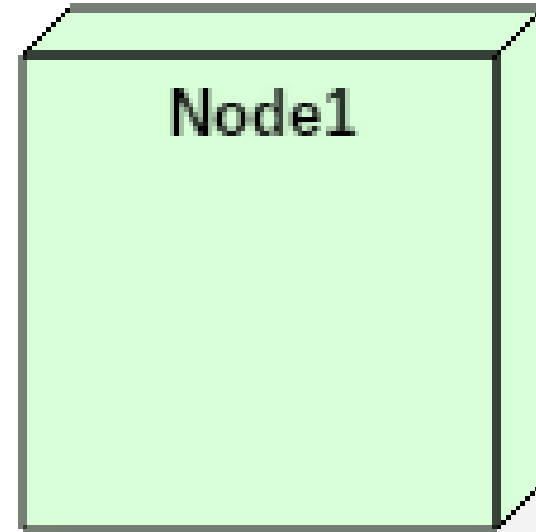
Deployment Diagram: Notations



Artifact1



Interface1



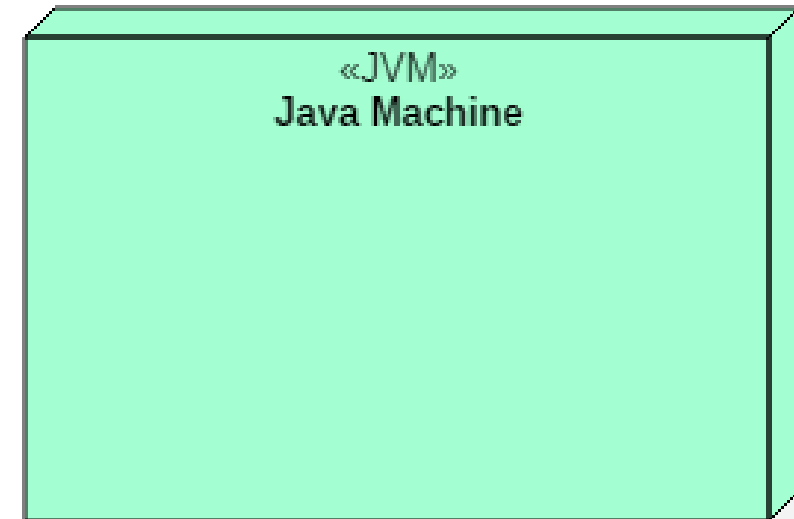
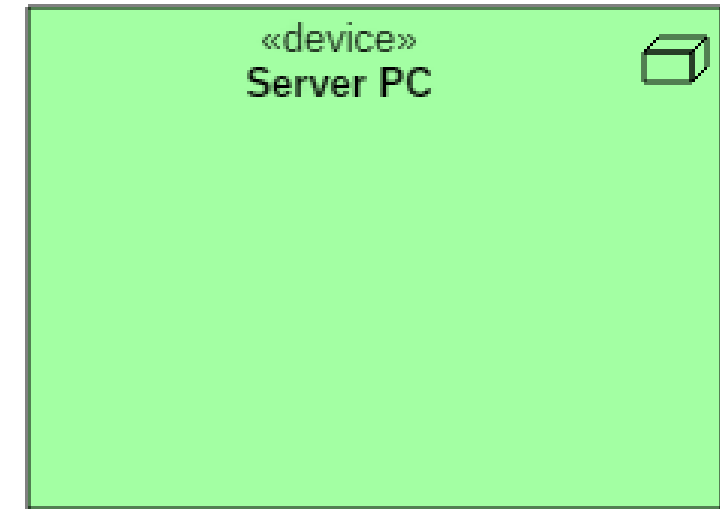
Deployment Diagram: Artifact



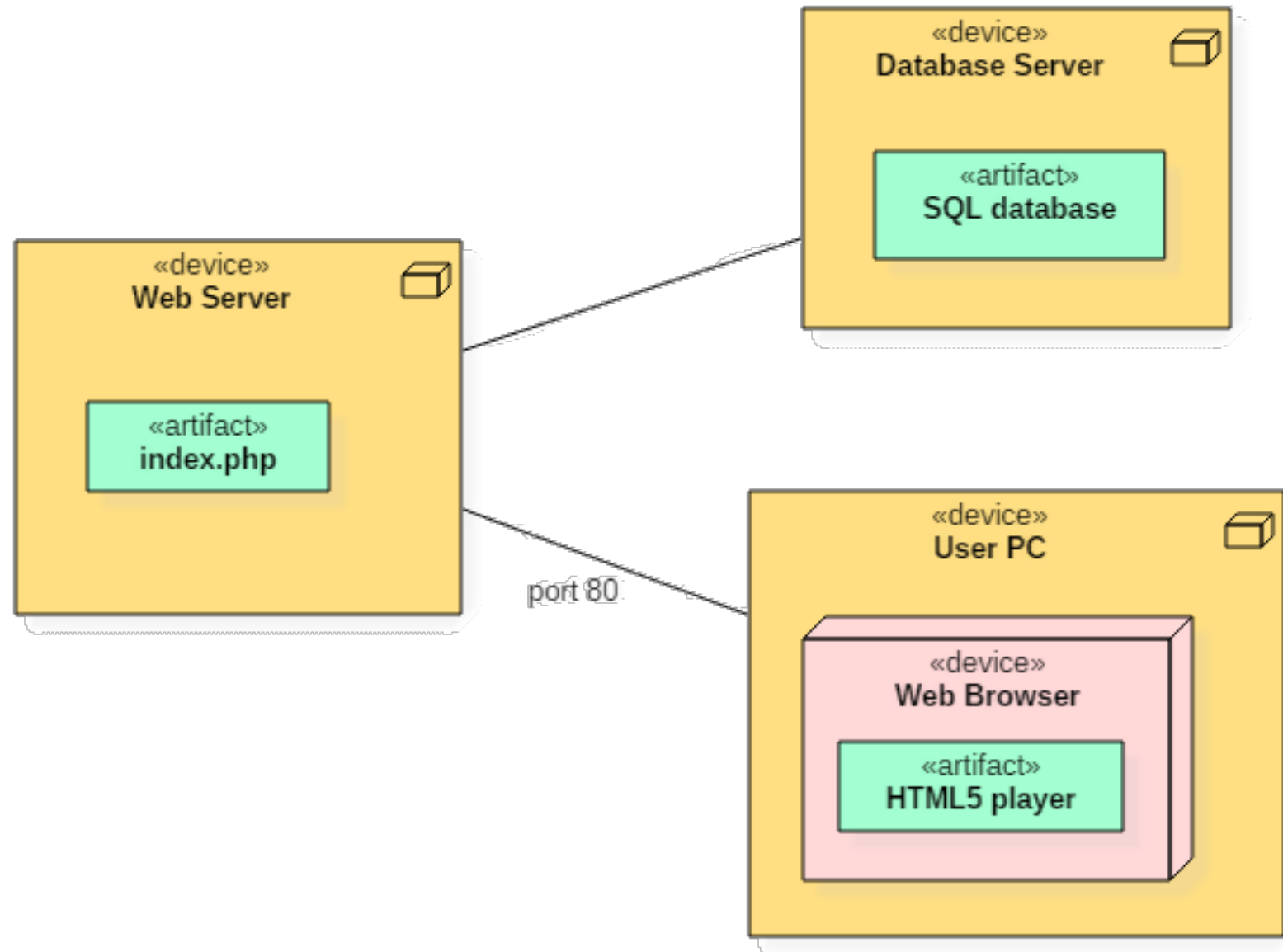
- Represent physical entities that are used or produced in a software development process
- Artifacts are deployed on the nodes. The most common artifacts are as follows:
 - Source files
 - Executable files
 - Database tables
 - Scripts
 - DLL files
 - User manuals or documentation
 - Output files
- Artifacts are labeled with the stereotype **<<artifact>>**, and it may have an artifact icon on the top right corner.
- Each artifact has a filename in its specification

Deployment Diagram: Node

- Node is a computational resource upon which artifacts are deployed for execution.
- A node is a physical thing that can execute one or more artifacts.
- Shown using the stereotype **<<device>>** or **<<execution environment>>**



Deployment Diagram Example: Working of HTML5 video player



Example Quiz



Match the file to the corresponding Deployment Diagram parts – Artifact or Node

BackendAPI

Artifact

Google App Engine

Node

SmartPhone

Node

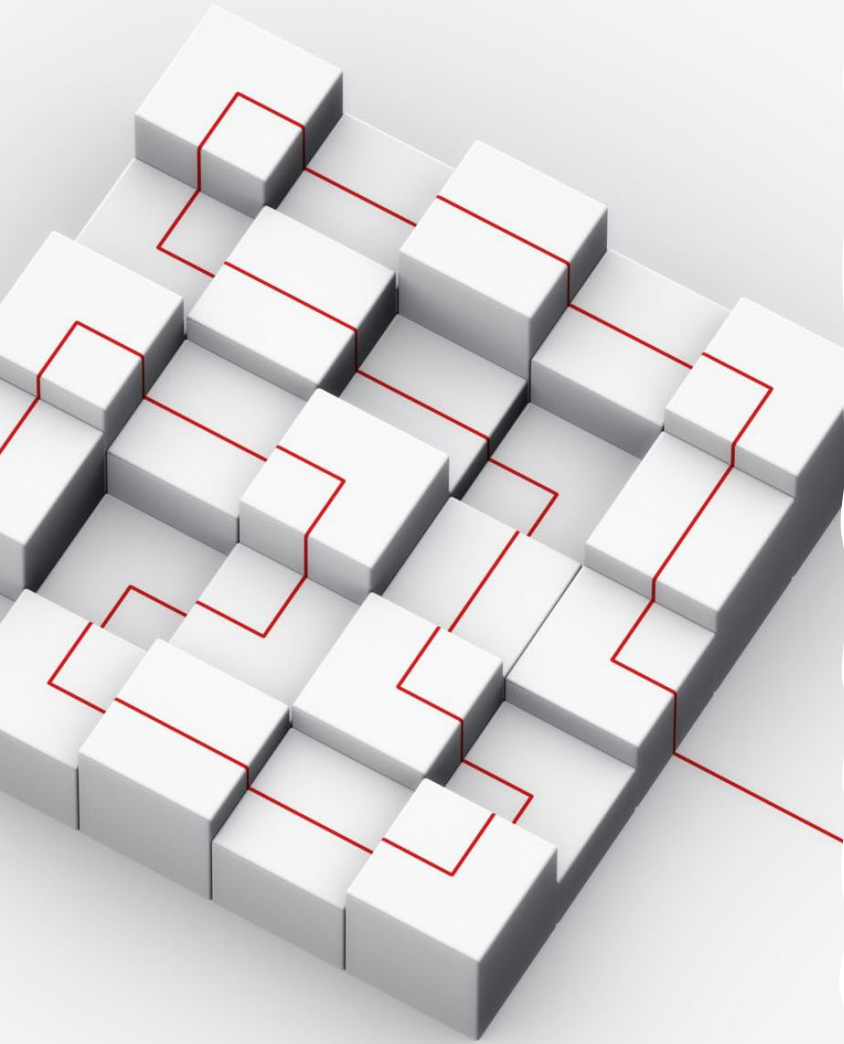
WebApp

Artifact



You need to create a component(or package diagram) and a deployment diagram for your project 1.





Low Level Design View – Structural Diagram: DCD

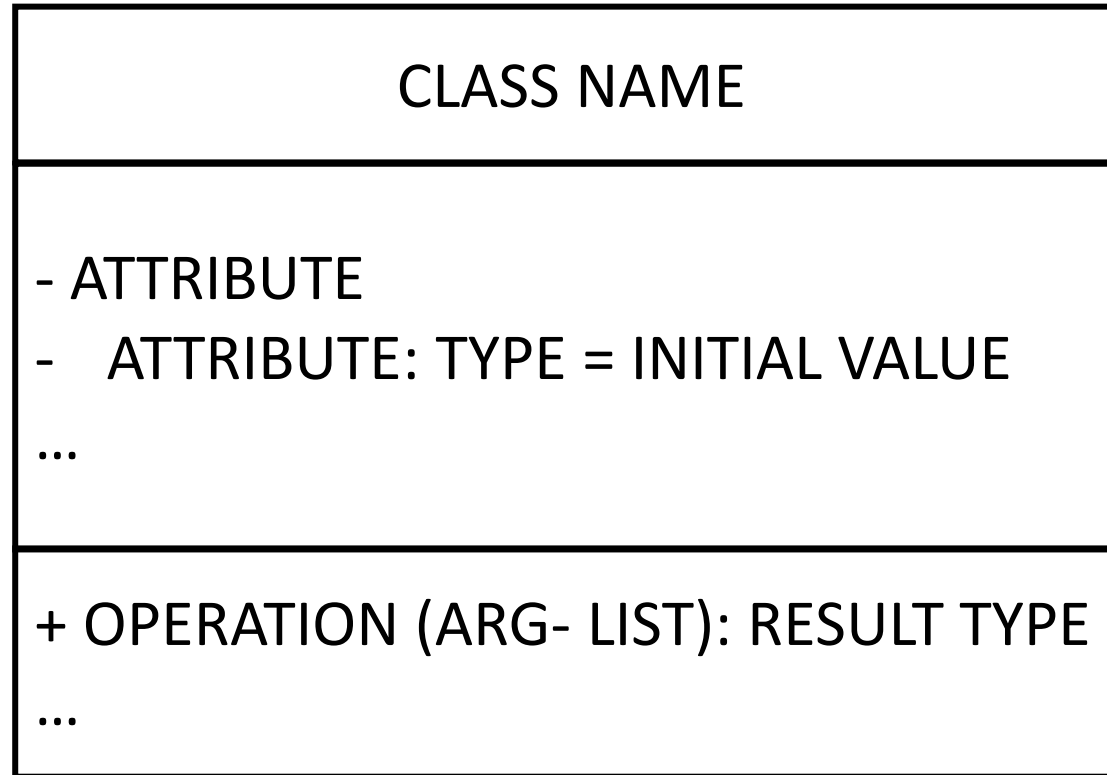
Design Class Diagram

Static, Structural View of the System

Describes

Classes and their Structure
Relationships among classes

Class Diagram: Class



Class names are identified as potential nouns from the requirements

Relationships in DCD

Generalization: X **is a** Y



Inheritance between classes or
interface implementation

Realization: X **is a** Y



interface implementation

Associations: X and Y
are related (**has**)



Can be dependency, aggregation, or
composition association

Can be unidirectional

Dependencies: X **uses** Y



methods of a class that use another class's
object as a parameter .

Aggregations: X **has** a Y



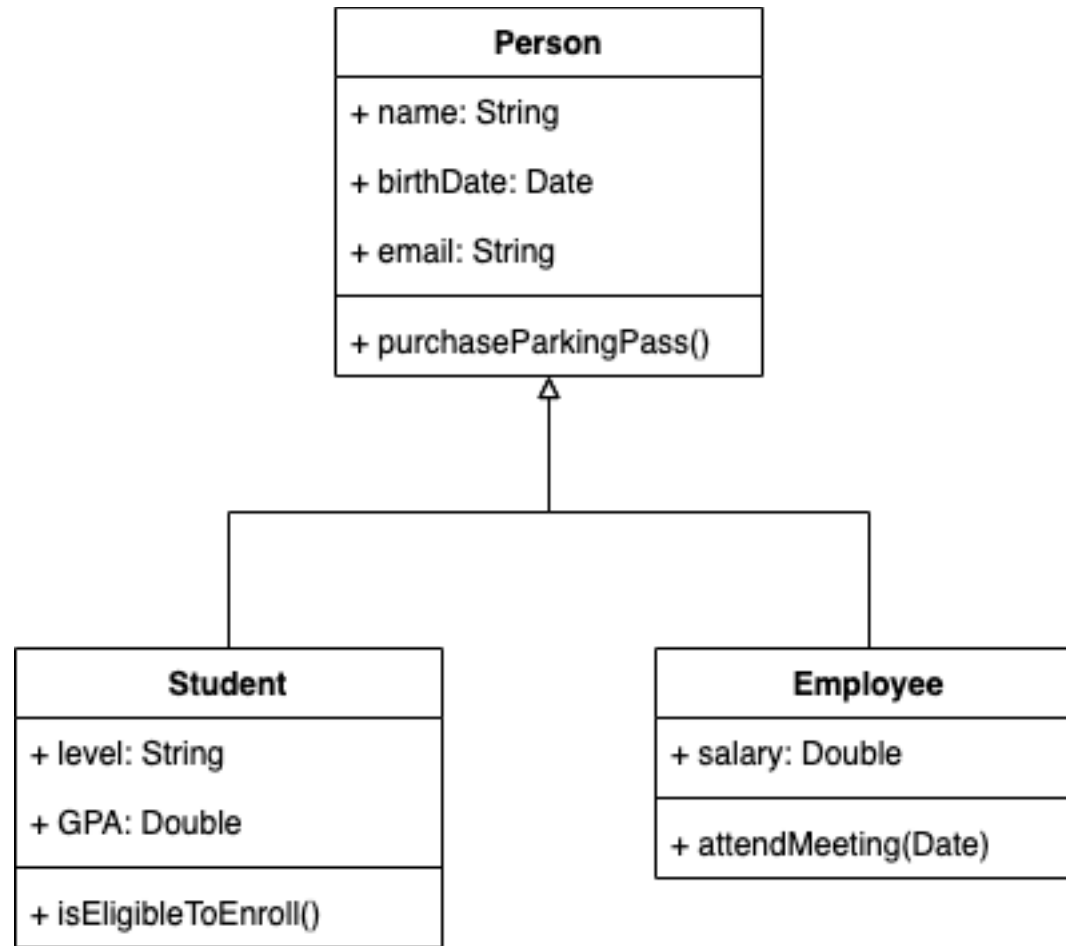
The relationship between the whole and the
part (parent and child), but the part can exist
independent of the whole

Composition: X **has** a Y

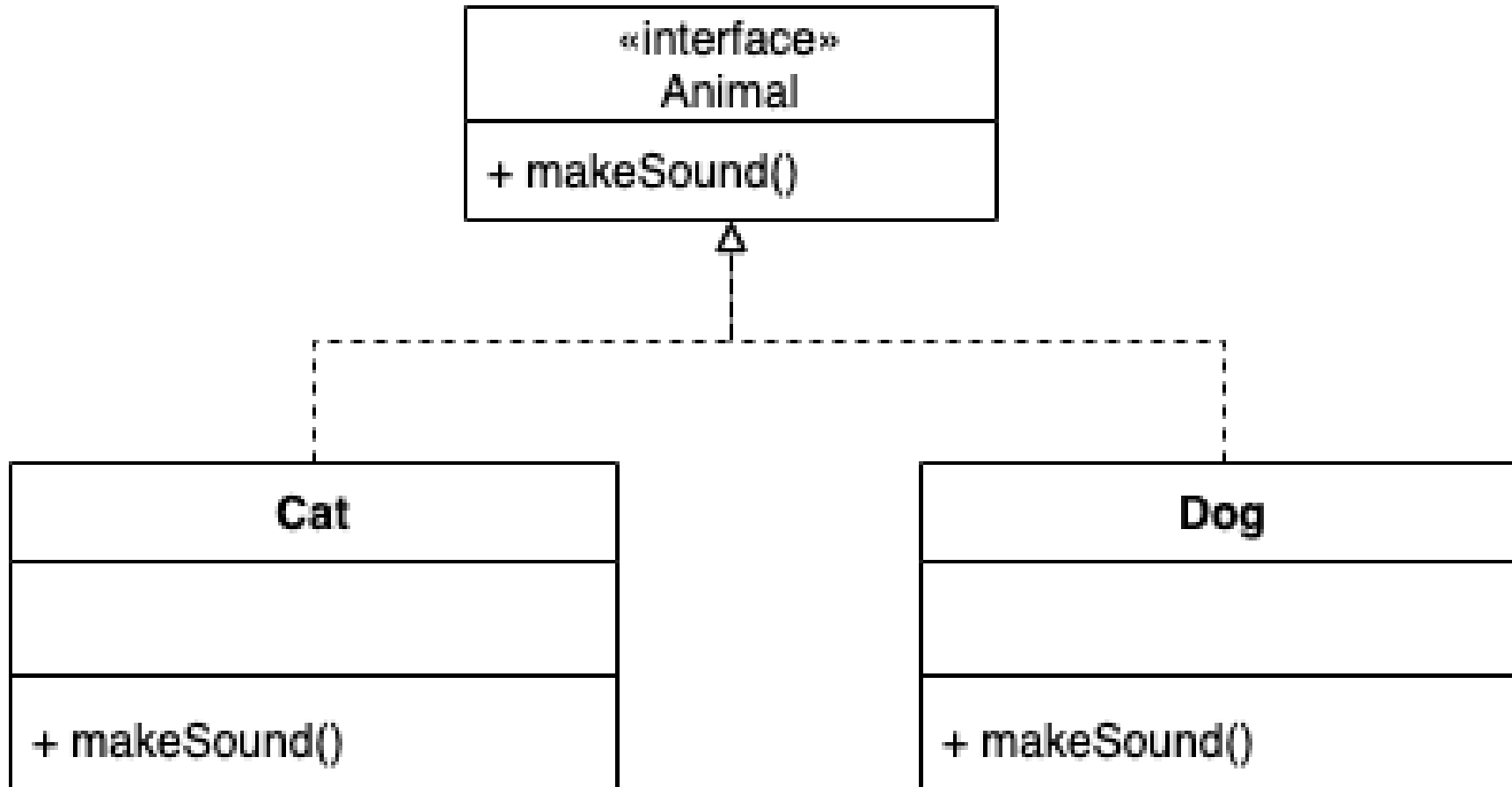


The relationship between the whole and
the part, but the part cannot exist
independent of the whole.

Generalization Example

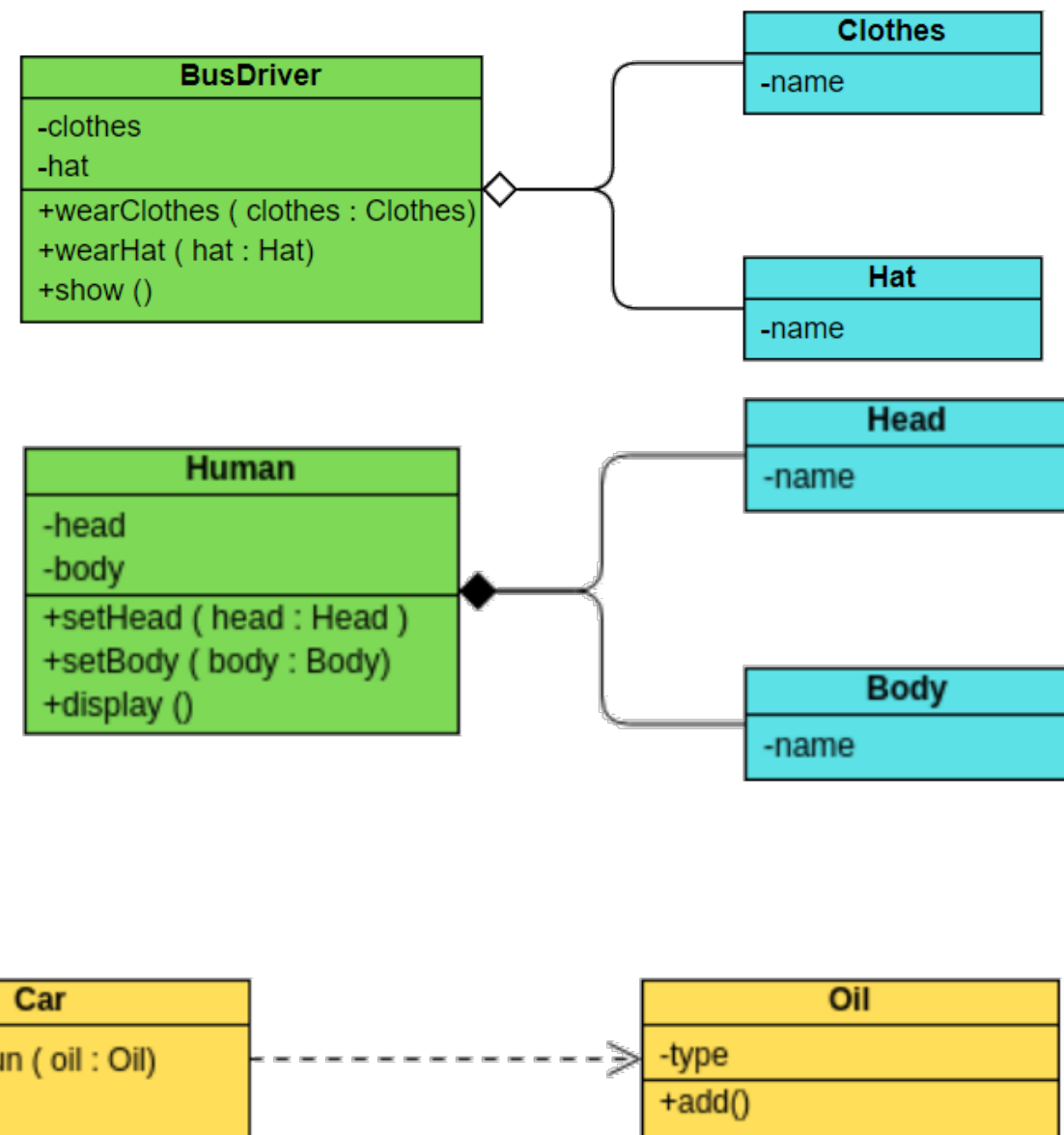


Generalization (Realization) Examples: Interface



Association Types

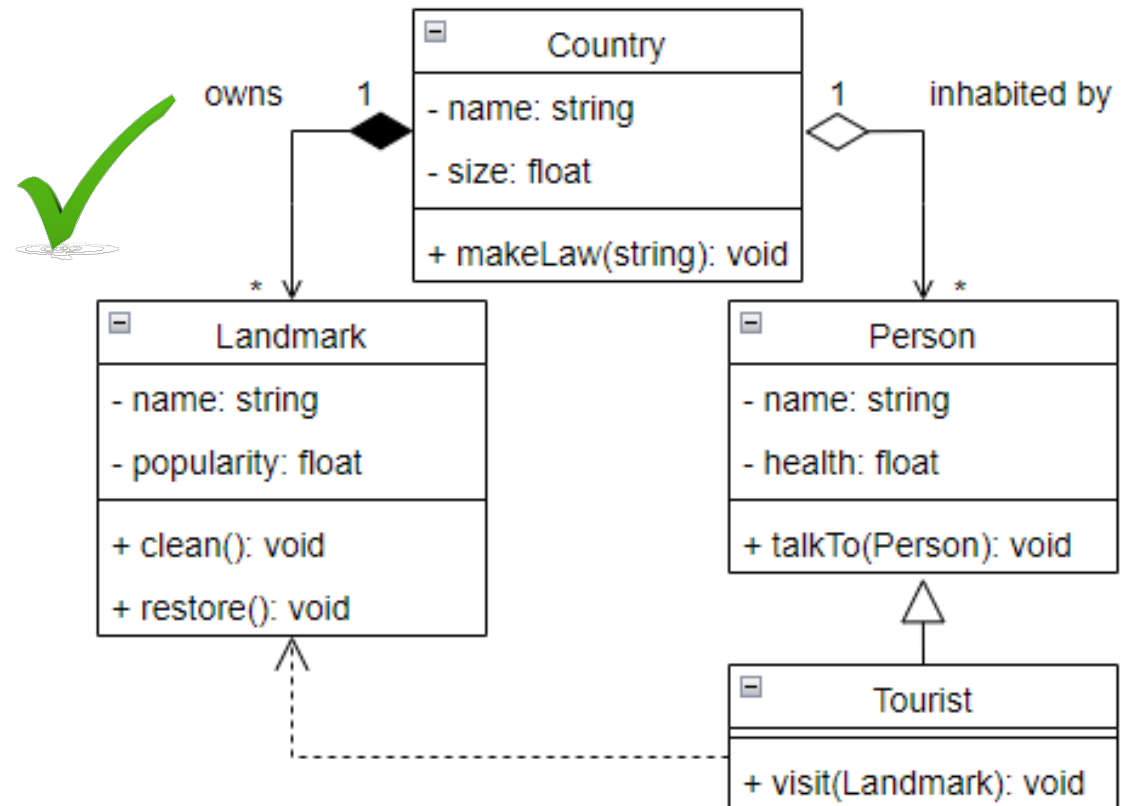
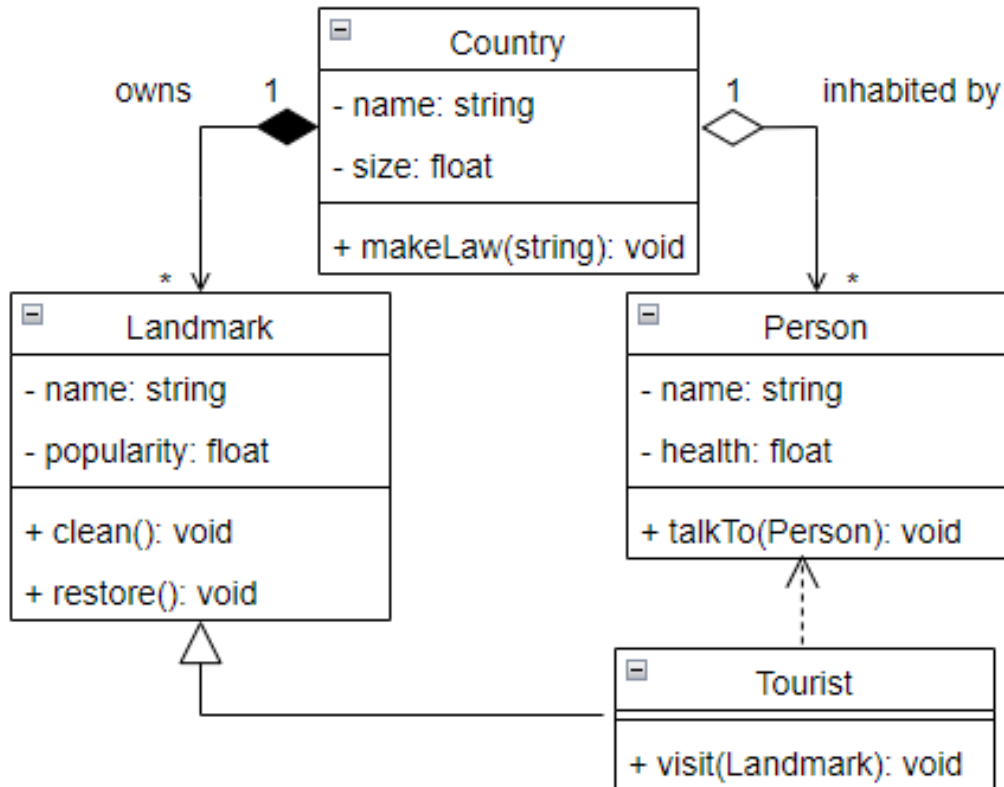
- aggregation: “is part of”
 - child can exist independently of the parent
- composition: “is entirely made of”
 - stronger version of aggregation
 - the parts live and die with the whole
 - child cannot exist independent of the parent
 - symbolized by a black diamond
- dependency: “uses temporarily”
 - symbolized by dotted line
 - often is an implementation detail, not an intrinsic part of that object's state

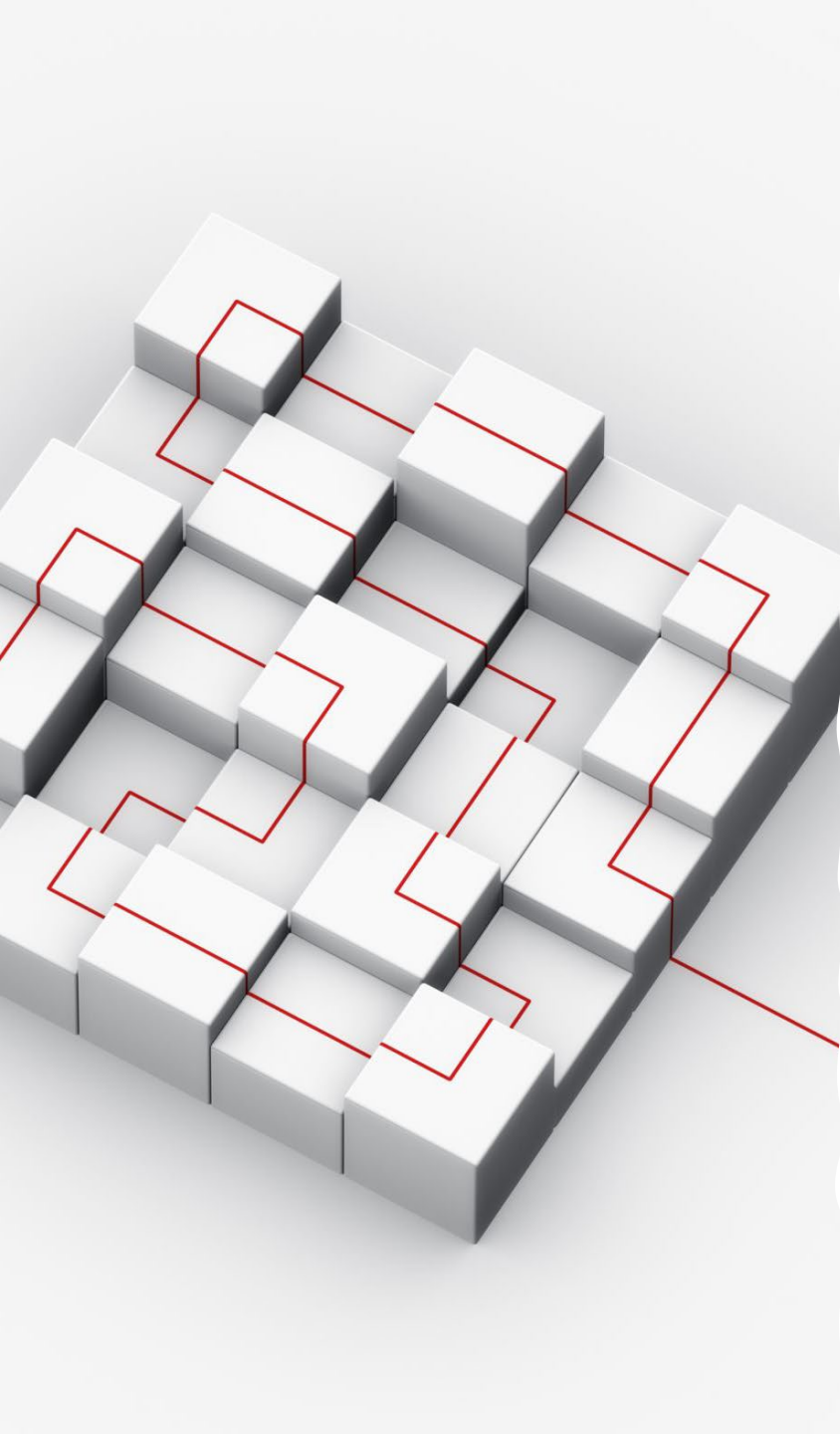




Example Quiz

Scenario: You are developing an application that simulates public travel trends. Each country in the system has a name, size, and can write laws. Each country also owns different landmarks, which have different names and popularity levels. The landmarks must be cleaned and restored over time, and **cannot move between countries**. Countries are also inhabited by people, who have a name and health level. They are able to talk to other people. Tourists are a type of person, with the additional ability to visit landmarks. Which is the correct DCD?





Low Level Design View – Behavioral Diagram: Use Case Diagram

Use Case Diagram: Structure

- actors as stick-figures, with their names (nouns)
- use case goals as ellipses with their names (verbs)
- line associations, connecting an actor to a use case in which that actor participates

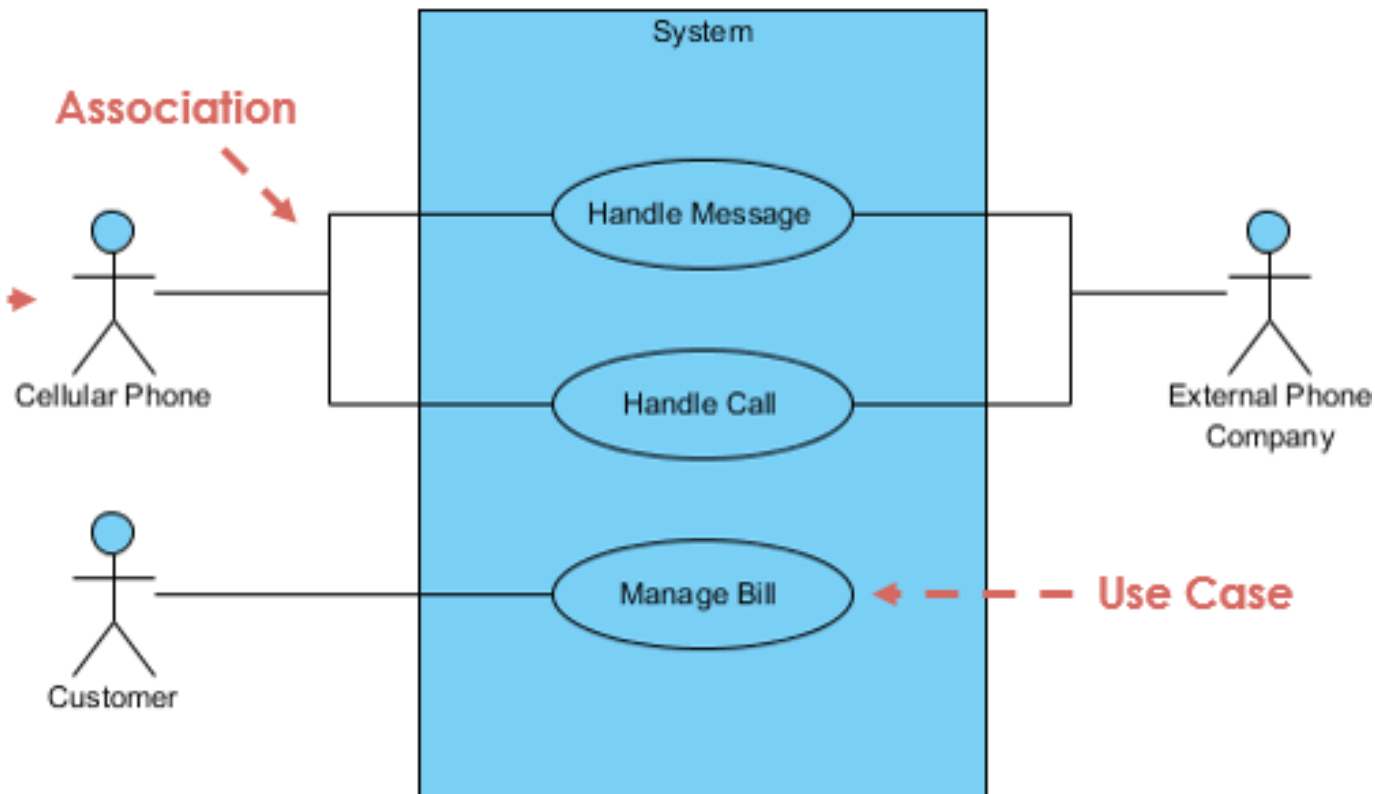
Primary actors to the left

System Boundary

Supporting actors to the right: they provide a service.

Actor - - - - ->

Association



Offstage actor towards the bottom

Use Case Relationships

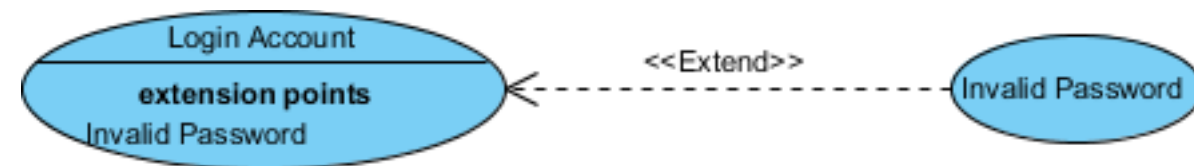
<<Include>>

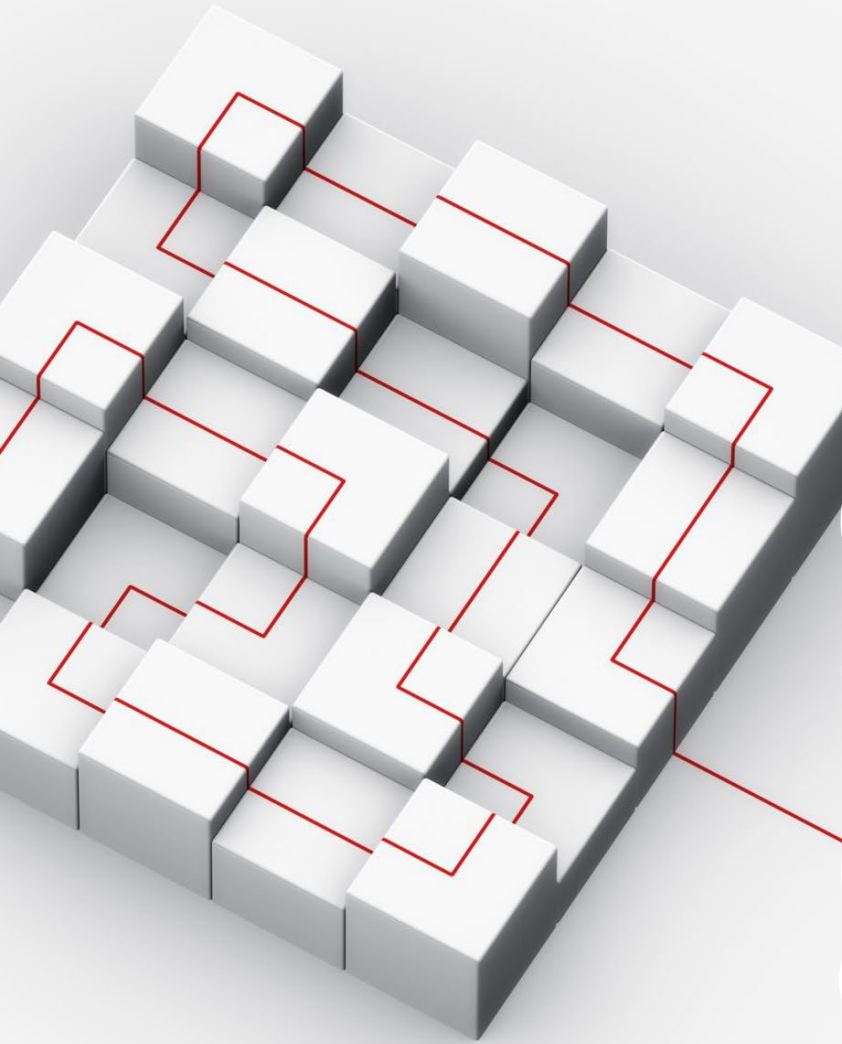
- Used when one use case is **used by** another use case
- Typically used if a sub-use case or series of steps is used by **several use cases**
- A **uses** relationship from base use case to child use case indicates that an instance of the base use case will **include** the behavior in the child use case.
- Depicted with a directed arrow having a dotted line. The tip of arrowhead **points to the child use case** and the parent use case connected at the base of the arrow.



<<Extend>>:

- Used when a use case may optionally take an alternate path
- You can think of these as exceptions to the typical path in the use case
- Depict with a directed arrow having a dotted line. The tip of arrowhead **points to the base use case** and the child use case is connected at the base of the arrow.
- The stereotype "<<extends>>" identifies as an extend relationship
- The extension point is specified in the base use case

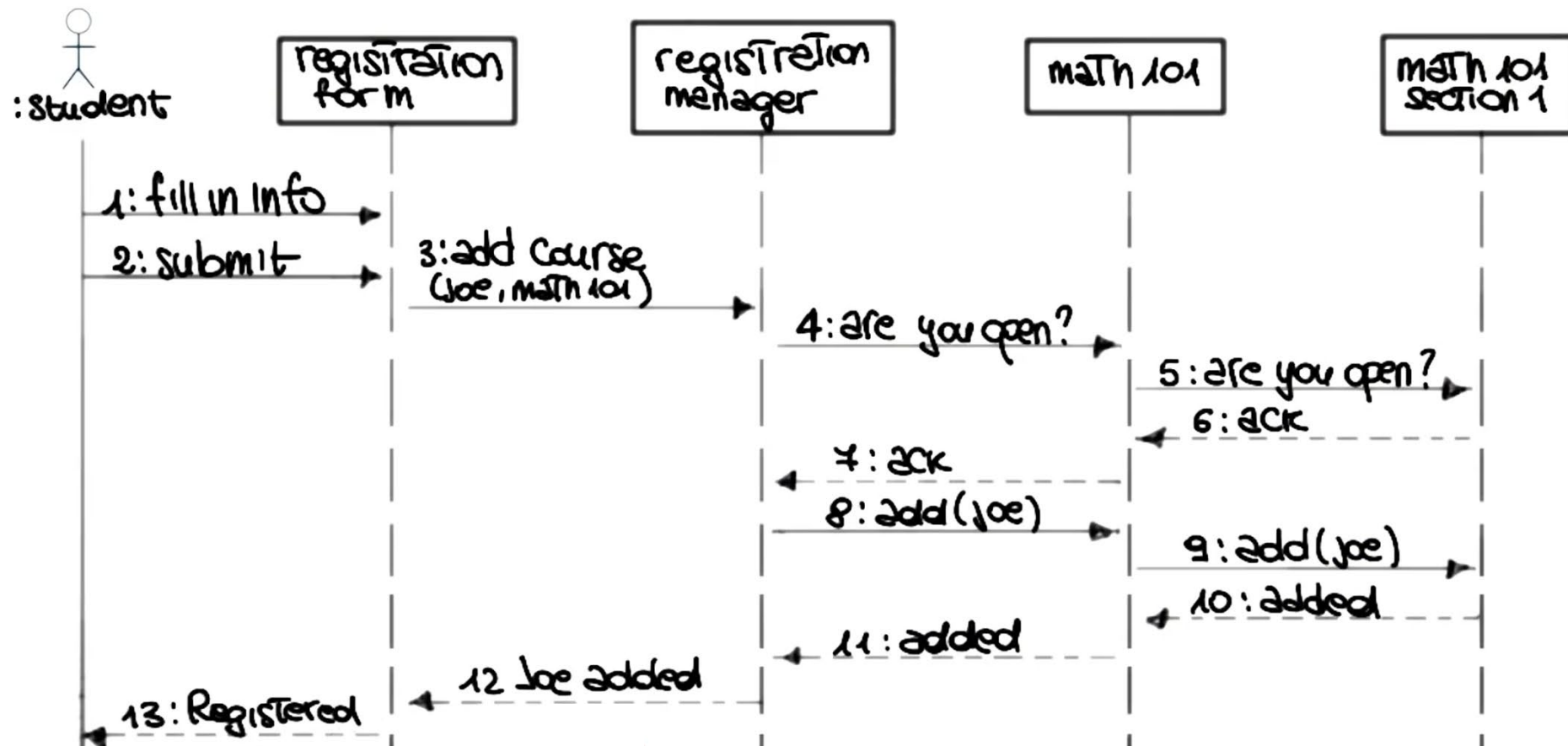




Low Level Design View – Behavioral Diagram: SD

Sequence Diagrams

Diagrams that emphasize time ordering of messages between classes/components





You need to
create a DCD,
UCD and SD for
your project 1.



AI considerations

- guidance for using AI tools like Lucidchart to create UML diagrams on Canvas and website
- Progressive specific prompting may be needed
- Manual edits maybe needed
- For the assignment, in each of the 4 compulsory diagrams, you need to compare the manually generated output vs. the 2 AI tool-generated outputs using the following metrics - Usability/User-friendliness, Autocompletion Rate (how much of the generation was AI vs. manual tweaks to the AI-generated diagram), Setup Time/Learning Curve, Number of Prompt Iterations/Refinement, Total Time Taken.